1. 文档结构介绍

首先, 第 2 章会给出整个设备模型的整体框架.

然后, 第 3 章介绍 sysfs, 了解它有助于我们理解设备模型到底在做什么.

接着, 第 4, 5, 6 章描述设备模型的基础概念, 驱动开发中基本上不会看到这些基础, 不过我们必须先理解它们, 才能更好的理解后续内容.

然后, 第 7 章简单介绍了下/sys/下的一些顶层目录是如何创建出来的, 可以浏览一下.

接着, 第 8, 9, 10, 11 章描述了设备模型中最重要的 4 个概念: Bus, Class, Device, Driver.

第 12 章介绍了 platform 系统, 它是基于 Bus, Class, Device, Driver 抽象出来的一个更上层的东西, 理解了前面的内容之后, 再来看 platfrom 系统就会非常容易了.

最后一章介绍了 devres, 它很容易理解, 也很实用. 教你在驱动开发过程中, 如何省去"goto 释放 resource 的烦恼".

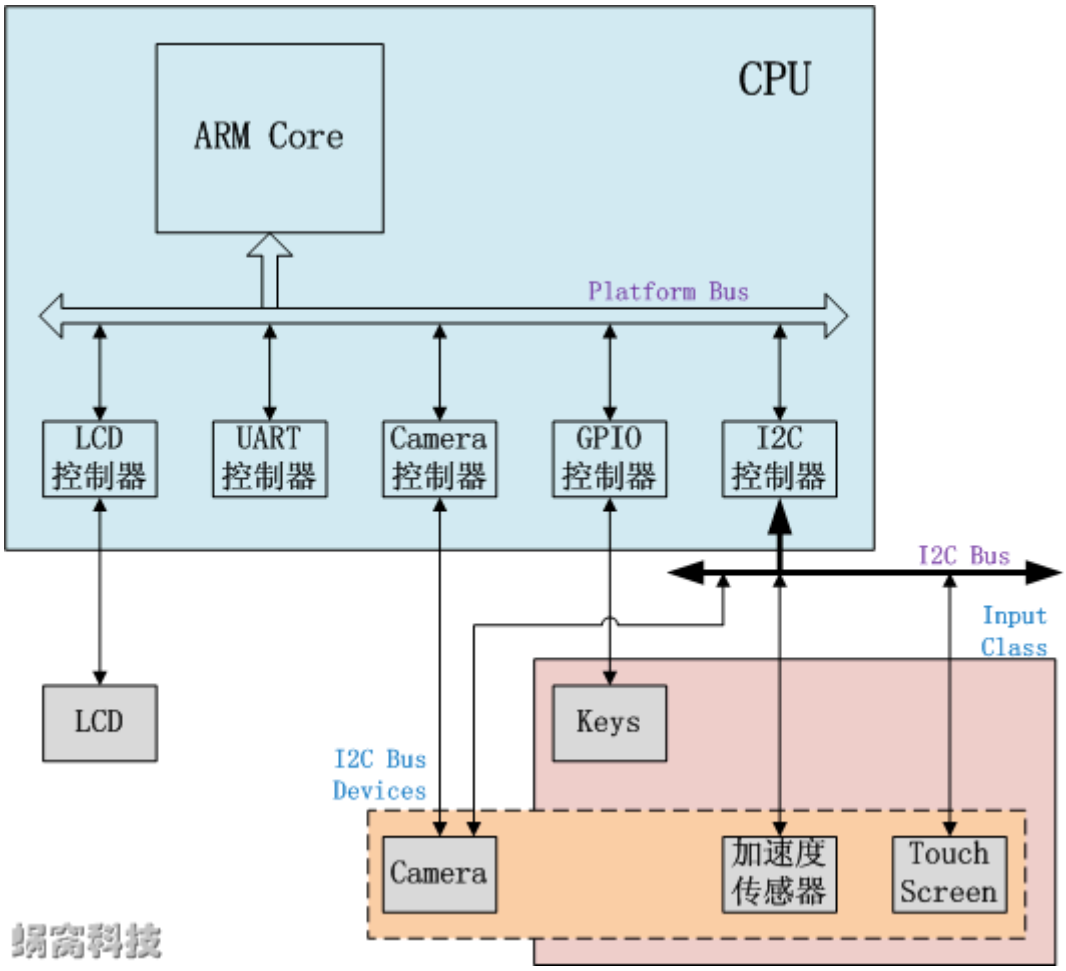
2. 架构介绍

由于 Linux 支持世界上几乎所有的、不同功能的硬件设备（这是 Linux 的优点）, 导致 Linux 内核中有一半的代码是设备驱动, 而且随着硬件的快速升级换代, 设备驱动的代码量也在快速增长. 为了降低设备多样性带来的 Linux 驱动开发的复杂度, 以及设备热拔插处理、电源管理等, Linux 内核提出了设备模型（也称作 Driver Model）的概念. 设备模型将硬件设备归纳、分类, 然后抽象出一套标准的数据结构和接口. 驱动的开发, 就简化为对内核所规定的数据结构的填充和实现.

2.1 设备模型的基本概念

Bus, Class, Device 和 Device Driver 的概念

下图是嵌入式系统常见的硬件拓扑的一个示例：



硬件拓扑描述 Linux 设备模型中四个重要概念中三个：Bus, Class 和 Device（第四个为 Device Driver, 后面会说）:

- Bus（总线）

  Linux 认为, 总线是 CPU 和一个或多个设备之间信息交互的通道. 而为了方便设备模型的抽象, 所有的设备都应连接到总线上

- Class（分类）

  在 Linux 设备模型中, Class 的概念非常类似面向对象程序设计中的 Class（类）, 它主要是集合具有相似功能或属性的设备, 这样就可以抽象出一套可以在多个设备之间共用的数据结构和接口函数. 因而从属于相同 Class 的设备的驱动程序, 就不再需要重复定义这些公共资源, 直接从 Class 中继承即可

- Device（设备）

  抽象系统中所有的硬件设备, 描述它的名字、属性、从属的 Bus、从属的 Class 等信息

- Device Driver（驱动）

  Linux 设备模型用 Driver 抽象硬件设备的驱动程序, 它包含设备初始化、电源管理相关的接口实现. 而 Linux 内核中的驱动开发, 基本都围绕该抽象进行（实现所规定的接口函数）

3. sysfs

快速体验什么是 sysfs

非常感性的一个认识是, 随便找个 Linux 的机器或者板子, 然后看看它的根目录(/)下是不是有个叫 sys 的文件夹? 对, /sys/目录就是这里所说的 sysfs.

sysfs 有什么用

ls /sys/ , 会看见几个熟悉的文件夹, bus, class, devices. 这几个文件夹是不是与设备模型中的几个概念名字一样. 猜想 sysfs 会不会跟设备模型有什么关系? 没错, sysfs 与设备模型关系紧密.

sysfs 的作用主要体现在两个方面, 从内核与用户空间的角度来说, 这两方面体现为:

- sysfs 把**内核的某些东西展示给用户空间**: 它展示设备模块中各个组件的**层次**关系, 层次关系就是一个树形结构. 顶层目录, 子目录, 子目录的子目录, 属性文件等. 用户空间可以用 ls 读取这些目录, 或者用 cat 读取属性文件.
- sysfs 可以**从用户空间得到一些信息传递给内核**: sysfs 的某个目录下可能会存在一些文件, 我们把这个文件称作**属性**文件. 用户空间可以修改这些属性文件, 文件被修改之后, sysfs 能够通过某种机制通知内核.

  Note: 还记得在内核模块一文, 我们曾经介绍过, 内核模块可以有模块参数, 我们也可以通过 sysfs 来读取/修改这些模块参数. 但是内核模块并不会收到参数被修改的通知, 模块只有主动读取参数, 与之前的值做对比, 才知道参数是否被修改.

  这里会有一个疑问, 模块参数也是通过 sysfs 修改的,为什么没有收到修改通知? 具体原因没有去细看. 不过我们会在下面的章节中介绍 sysfs 是通过什么方式通知内核某个属性文件被修改了. 了解这种机制之后, 如果有兴趣追踪上面的疑问, 可以自行阅读模块参数的源码细节.

详解了解 sysfs

sysfs 是一种基于 RAM 的虚拟文件系统

简单说明一下什么是虚拟文件系统.

要说明虚拟文件系统, 首先来看一下什么是文件系统. FAT/FAT32, 这是最常见的文件系统, 我们的 U 盘一般就是用的这种文件系统. 除了这两种, 还有众多的文件系统: EXT2, EXT4, JIFFS2, UBIFS …

Linux 内核为了给用户空间一个统一的接口, 在这些真实的文件系统之上, 封装了一层虚拟文件系统(VFS). 这样用户空间看到的就是 VFS 提供的统一接口, 而真实文件系统之间的差异, 则在封装层由内核屏蔽掉了, 用户空间不用去关注这些差异. 通过内核提供的 register_filesystem 函数, 就能向 VFS 注册一个虚拟的文件系统.

sysfs 就是一种虚拟的文件系统, 它的代码路径在 fs/sysfs. 它的初始函数为 sysfs_init, 在该函数中, 会调用 register_filesystem, 向 VFS 注册虚拟文件系统.

说它基于 RAM 的意思是, sysfs 是构建在 RAM 上的, 用户空间所有对 sysfs 的增/删/修改操作, 重启之后都没了. 比如你在 sysfs 下创建了一个文件, 重启之后, 这个文件就没了.

sysfs proc devfs udev devtmpfs 的区别

首先, 除了 udev, 其他几个都是虚拟文件系统.

devfs, 是在 Linux 2.4 引入的, 它叫设备文件系统, 主要与/dev 目录下的设备节点有关. 曾经被许多工程师给予高度评价. 不过在 Linux 2.6 中被 udev 替代了, 原因是 devfs 所做的工作被确信可以在用户态来完成, 而且 devfs 的维护者停止了对代码的维护.

udev, Linux 2.6 中引入, 完全在用户态工作, 它是一个 daemon, 利用设备加入或移除时内核所发送的热插拔事件来管理/dev 目录下设备节点的创建和删除操作. 本文后面会有一个章节来专门介绍内核是如何发送热插拔事件的.

devtmpfs, 可以被理解为一个增强版的 devfs, 它在设计之初经历了一些争论, 有人赞同, 有人反对, 细节参考这里. 不过最终还是在 Linux 2.6.31 时加入内核. 它的主要目的是优化启动时间, 某些嵌入式设备对启动时间要求较高, 而 udev 由于某些原因会拖慢启动时间. devtmpfs 最终会被 mount 到/dev 目录下.

proc, 用于提供进程和状态信息的虚拟文件系统

sysfs, Linux 2.6 中引入, 用来展示设备模型中的各个部分之间的层级关系, 属性.

为什么要在这里介绍这几种文件系统, 一方面是对 sysfs 有个对比了解, 另外一方面, 本文后面章节在讲述 uevent 事件的时候, 需要本小节的背景知识.

4. kobject & ktype & attribute

4.1 简介

Kobject

一个感性的认识是: kobject 对应/sys/下的一个目录(文件夹)或子目录. 但不是/sys/下所有的目录都对应 kobject, 有的目录是以其他形式创建的.
Kobject 是 Linux 设备模型的基石:
Linux 设备模型的核心是使用 Bus、Class、Device、Driver 四个核心数据结构, 将大量的、不同功能的硬件设备（以及驱动该硬件设备的方法）, 以树状结构的形式, 进行归纳、抽象, 从而方便 Kernel 的统一管理;
而硬件设备的数量、种类是非常多的, 这就决定了 Kernel 中将会有大量的有关设备模型的数据结构. 这些数据结构一定有一些共同的功能, 需要抽象出来统一实现, 否则就会不可避免的产生冗余代码;
这就是 Kobject 诞生的 2 个背景, 因此 Kobject 的主要作用也可以归纳为 2 方面.

Kobject 主要有 2 方面的作用:

- 以**树形结构**组织层次: 通过 parent 指针, 可以将所有 Kobject 以层次结构的形式组合起来
- **引用计数**: 使用一个引用计数（reference count）, 来记录 Kobject 被引用的次数, 并在引用次数变为 0 时把它释放

Note1: 在 Linux 中，Kobject 几乎不会单独存在。它的主要功能，就是内嵌在一个大型的数据结构中，为这个数据结构提供一些底层的功能实现
Note2: Linux driver 开发者，很少会直接使用 Kobject 以及它提供的接口，而是使用构建在 Kobject 之上的设备模型接口

Attribute

一个感性的认识是: Attribute 就是/sys/某个目录下的一个文件, 这个文件叫属性文件.
Attribute 与 Kobject 的关系也很明显, 文件是存在于某个目录下的, 所以 Attribute 也是跟 Kobject 挂钩的. 我们一般说某个 Kobject 具有某些属性.

Attribute 有两个主要参数:

name: 名字, /sys/下的文件名, 就是这个 name

mode: 权限, 该文件的读写权限

ktype

没办法给 Ktype 一个感性的认识.
Ktype 主要作用是把 Kobject 的一些通用的东西抽象到一起.
比如我们在上面提到, Kobject 会提供引用计数的功能, 当计数到 0 时, 就会释放资源, 这样每个 Kobject 都应该有一个 release 的方法来释放资源

在比如上面提到的属性文件, 有可能几个 Kobject 都具有相同的属性, 我们就可以把这些属性定义在一个 Ktype 里面

Ktype 的概念在内核里面比较难得理解, 我们会在本章的最后, 对 Kobject, Ktype, Attribute 做一个整体的说明, 以便于理解.

综合上文描述的, Ktype 主要有以下几方面的作用:

- **release**: Kobject 的资源释放方法
- struct attribute ***default_attrs** : 注意它是一个双指针, 相当于一个链表头. 每个 Attribute 对应一个属性文件, 链表头代表着会有多个属性文件. 当某个 Kobject 指向该 Ktype 时, 这个 Kobject 就会包含所有的这些属性文件.
- **sysfs_ops** : 属性文件对应的读/写函数

4.2 主要数据结构

Kobject

/* Kobject: include/linux/kobject.h line 60 */

struct kobject	Comment
const char *name	该 Kobject 的名称, 同时也是 sysfs 中的目录名称
struct list_head entry	用于将 Kobject 加入到 Kset 中的 list_head (Kobject 与 Kset 的关系下章详解)
struct kobject *parent	指向 parent kobject, 以此形成层次结构（在 sysfs 就表现为目录结构）
struct kset *kset;	该 kobject 属于的 Kset. 可以为 NULL. 如果存在, 且没有指定 parent, 则会把 Kset 的（因为 Kset 是一个特殊的 Kobject）
struct kobj_type *ktype;	该 Kobject 属于的 kobj_type. 每个 Kobject 必须有一个 ktype, 否则 Kernel 会提示错
struct sysfs_dirent *sd;	该 Kobject 在 sysfs 中的表示
struct kref kref;	“struct kref”类型（在 include/linux/kref.h 中定义）的变量, 为一个可用于原子操
unsigned int state_initialized:1	指示该 Kobject 是否已经初始化, 以在 Kobject 的 Init, Put, Add 等操作时进行异常
unsigned int state_in_sysfs:1	指示该 Kobject 是否已在 sysfs 中呈现, 以便在自动注销时从 sysfs 中移除
unsigned int state_add_uevent_sent:1	记录是否已经向用户空间发送 ADD uevent
unsigned int state_remove_uevent_sent:1	记录是否已经向用户空间发送 REMOVE uevent, 如果已经发送了 ADD uevent, 上 uevent, 则在自动注销时, 补发 REMOVE uevent, 以便让用户空间正确处理
unsigned int uevent_suppress	如果该字段为 1, 则表示忽略所有上报的 uevent 事件

Note: Uevent 提供了“用户空间通知”的功能实现, 通过该功能, 当内核中有 Kobject 的增加、删除、修改等动作时, 会通知用户空间.

有关该功能的具体内容, 会在其本文后面章节详细描述。

Attribute & bin_attribute

/* attribute: include/linux/sysfs.h */

struct attribute 为普通的 attribute, 使用该 attribute 生成的 sysfs 文件, 只能用字符串的形式读写. 一般这种形式用于内核把某个变量暴露给用户空间, 以便用户空间修改此变量

struct attribute	Comment
const char *name	该 Attribute 的名称, 同时也是 sysfs 中的文件名称
umode_tmode	读/写权限
#ifdef CONFIG_DEBUG_LOCK_ALLOC boolignore_lockdep:1; struct lock_class_key*key; struct lock_class_keyskey; #endif	CONFIG_DEBUG_LOCK_ALLOC 相关, 这里不关注

struct bin_attribute 为二进制属性, 在 struct attribute 的基础上, 增加了 read、write 等函数,它所生成的 sysfs 文件可以用任何方式读写. 一般这种形式用于把某块 Memory 暴露给用户空间, 比如把 EEPROM 的存储空间暴露给用户空间, 在用户空间直接读/写 EEPROM.
这种属性文件本文暂不做详细分析.

struct bin_attribute	Comment
struct attributeattr	封装 struct attribute
size_tsize;	
void*private	
ssize_t (*read)(struct file *, struct kobject *, struct bin_attribute *,char *, loff_t, size_t)	读函数
ssize_t (*write)(struct file *, struct kobject *, struct bin_attribute *,char *, loff_t, size_t)	写函数

int (*mmap)(struct file *, struct kobject *, struct bin_attribute *attr, struct vm_area_struct *vma)	map 函数

Kobj_attribute

/* kobj_attribute: include/linux/kobject.h line 138 */

struct kobj_attribute	Comment
struct attribute attr	表明是对 struct attribute 结构体的封装
show	属性文件读函数 是不是有点迷茫了! 属性文件的读/写操作不是在 ktype 中定义的吗? 这里又是 完, 在本章最后一小节"总结"中会阐释其中的关系
store	属性文件写函数

Ktype

/* include/linux/kobject.h, line 108 */

struct kobj_type	Comment
void (*release)(struct kobject *kobj)	通过该回调函数, 可以将包含该种类型 kobject 的数据结构的内存
const struct sysfs_ops *sysfs_ops	该种类型的 Kobject 的 sysfs 文件系统接口, 也就是属性文件的读/
struct attribute **default_attrs	该种类型的 Kobject 的 atrribute 列表（所谓 attribute, 就是 sysfs 文 件）. 将会在 Kobject 添加到内核时, 一并注册到 sysfs 中
const struct kobj_ns_type_operations *(*child_ns_type)(struct kobject *kobj)	和文件系统（sysfs）的命名空间有关, 这里不再详细说明

const void *(*namespace)(struct kobject *kobj)	和文件系统（sysfs）的命名空间有关, 这里不再详细说明

4.3 主要 API 说明

Kobject

头文件: include/linux/kobject.h

实现代码: lib/kobject.c

kobject API	Comment
void kobject_init(struct kobject *kobj, struct kobj_type *ktype)	我们可以用 kmalloc 自行分配一个 Kobject 结构体, 然后用该 API 初始化此结构 注意在该 API 需要传递一个 kobj_type 类型的参数, 该参数是必须的, 不能为 提到的, 每个 Kobject 都需要有一个 Ktype 如果我们调用 kmalloc 自行分配了 Kobject, 那么我们也必须自行分配并初始化 之后的 Ktype 传给 Kobject
int kobject_add(struct kobject *kobj, struct kobject *parent, const char *fmt, ...)	将初始化完成的 kobject 添加到 kernel 中, 参数包括需要添加的 kobject、该 ko 于形成层次结构, 可以为空）、用于提供 kobject name 的格式化字符串.
int kobject_init_and_add(struct kobject *kobj, struct kobj_type *ktype, struct kobject *parent, const char *fmt, ...)	是上面两个接口的组合
extern void kobject_del(struct kobject *kobj);	add 对应的反函数, 从 kernel 中移除 kobject
extern struct kobject * __must_check kobject_create(void);	该 API 会分配一个 Kobject 内存空间, 然后调用 kobject_init 初始化 Kobject. 初始 是系统自己定义的一个叫 dynamic_kobj_ktype 的 Ktype. 如果我们使用这个 API, 那么我们不需要自己去分配 Kobject 空间, 也不 Ktype, 系统会自己处理好

	该 API 会返回一个初始化好的 Kobject, 接下来我们就可以调用 kobject_add 将
extern struct kobject * __must_check kobject_create_and_add(const char *name, struct kobject *parent);	Create 之后自动 add
kobject_set_name_vargs	设置 kobject 的 name
kobject_set_name	对上个函数的封装, 用于设置 kobject 的 name
kobject_rename	修改 kobject 的名字
kobject_get	调用 kref_get(&kobj->kref), 来增加 kobject 的引用计数. 一般我们在用 kobject_ 添加到内核时, 如果它的 parent 不为 NULL, 则会把 parent 指向的 kobject 的引
kobject_put	调用 kref_put(&kobj->kref, kobject_release)来减少 kobject 的引用计数. 当计数 去的 kobject_release 函数释放资源.

Attribute

头文件: include/linux/sysfs.h

实现代码: fs/sysfs/file.c

Attribute API	Comment
sysfs_create_file(struct kobject *kobj,const struct attribute *attr)	在某个 kobject 下创建 1 个属性文件

sysfs_remove_file(struct kobject *kobj, const struct attribute *attr)	删除 kobject 下的某个属性文件
sysfs_create_group(struct kobject *kobj,const struct attribute_group *grp)	在 kobject 下创建多个属性文件, attribute_group 相当于一个数组, 里面可以存放多 有意思的这个 group 还能有一个 name, 当 name 为 NULL 时, 所有的属性文件都会 的目录下; 当 name 不为 NULL 时, 会在 kobject 对应的目录下产生一个 name 目录, 存放在 name 目录下
sysfs_remove_group(struct kobject *kobj, const struct attribute_group *grp)	删除 kobject 下的某个属性组
sysfs_create_groups(struct kobject *kobj, const struct attribute_group **groups)	创建多个属性组
sysfs_remove_groups(struct kobject *kobj,const struct attribute_group **groups)	删除多个属性组

Kobj_attribute & __ATTR

头文件: include/linux/sysfs.h
这个算不上 API, 它是一个宏, #define __ATTR(_name, _mode, _show, _store) . 这个宏用于初始化 kobj_attribute 数据结构.

4.4 关键代码分析 & DEMO

想象这样一个场景: 我们想写一个内核模块, 加载到内核(还记得内核模块怎么写吧!), 用于在/sys/下创建一个目录, 然后在该目录下创建 1 个属性文件, 可以读/写该属性文件. 先不要看下面的内容, 想想你会怎么做?

首先, 我们需要用 kobject 创建一个目录, 创建目录的方式有两种:

- 手动创建: 手动创建的意思是用 kmalloc 自行分配 kobject, 定义 Ktype, 调用 kobject_init 初始化, 然后调用 kobject_add 添加到内核
- 自动创建: 自动创建的意思是用 kobject_create 或者 kobject_create_add 搞定所有事情.

然后, 我们需要在 kobject 目录下创建属性文件, 创建属性文件很多种方式:

- 如果是手动创建的 kobject 目录, 则
  - 可以自行定义属性文件
  - 也可以在 Ktype 里面定义默认的属性列表(参考 Ktype 数据结构), 当把 kobject 添加到内核时, 会自动出现这些属性文件.
- 如果是自动创建, 则
  - 只能通过调用 Attribute API 来添加属性文件, 不能定义默认属性列表. 原因是在自动创建时, 系统会在内部处理 Ktype, 我们没法添加默认属性列表.

手动创建 Kobject

分析几个关键函数

kobject_init

初始化通过 kmalloc 等内存分配函数获得的 struct kobject 指针. 主要执行逻辑为

- 确定参数 ktype 不为空, 也就是初始化 kobject 时, 一定要给它一个 Ktype
- 如果该指针已经初始化过（判断 kobj->state_initialized）, 打印错误提示及堆栈信息（但不是致命错误, 所以还可以继续）
- 调用 kobject_init_internal 初始化 kobj 内部的参数, 包括引用计数、list、各种标志等
  - 在初始化引用计数时, 调用的函数是 kref_init, **该函数会把引用计数值初始化为 1**.
- 根据输入参数, 将 ktype 指针赋予 kobj->ktype

kobject_add

将初始化完成的 kobject 添加到 kernel 中, 参数包括需要添加的 kobject、该 kobject 的 parent（用于形成层次结构, 可以为空）、用于提供 kobject name 的格式化字符串. 主要执行逻辑为:

- 确认 kobj 不为空, 确认 kobj 已经初始化, 否则错误退出
- 调用内部接口 kobject_add_varg, 完成添加操作

- kobject_add_varg, 解析格式化字符串, 将结果赋予 kobj->name, 之后调用 kobject_add_internal 接口, 完成真正的添加操作
  - kobject_add_internal: 将 kobject 添加到 kernel. 主要执行逻辑为
    - 校验 kobj 以及 kobj->name 的合法性, 若不合法打印错误信息并退出
    - 调用 kobject_get 增加该 kobject 的 parent 的引用计数（如果存在 parent 的话）
    - 如果存在 kset（即 kobj->kset 不为空）, 则调用 kobj_kset_join 接口加入 kset. 同时, 如果该 kobject 没有 parent, 却存在 kset, 则做两件事: 将 kobject 的 parent 设为 kset 的 kobject（kset 是一个特殊的 kobject）, 并增加 kset->kobjecct 的引用计数

      NOTE: 该章节暂不考虑 kset, kset 为 NULL, 下一章专门介绍 kset
    - 通过 create_dir 接口，调用 sysfs 的相关接口，在 sysfs 下创建该 kobject 对应的目录
    - 如果创建失败，执行后续的回滚操作，否则将 kobj->state_in_sysfs 置为 1

kobject_get

调用 kref_get, 增加引用计数

kobject_put

以系统定义接口 kobject_release 为参数, 调用 kref_put. kref 模块会在引用计数为零时, 调用 kobject_release

- kobject_release **:** 通过 kref 结构, 获取 kobject 指针, 并调用 kobject_cleanup 接口继续
- kobject_cleanup**:** 负责释放 kobject 占用的空间, 主要执行逻辑如下
  - 检查该 kobject 是否有 ktype, 如果没有, 打印警告信息
  - 如果该 kobject 向用户空间发送了 ADD uevent 但没有发送 REMOVE uevent, 补发 REMOVE uevent
  - 如果该 kobject 有在 sysfs 文件系统注册, 调用 kobject_del 接口, 删除它在 sysfs 中的注册
  - 调用该 kobject 的 ktype 的 release 接口, 释放内存空间
  - 释放该 kobject 的 name 所占用的内存空间

DEMO

https://gitlab.com/study-linux/linux_driver_model/blob/master/manual_kobject_attribute.c

NOTE1: 为什么要练习手动方式创建 kobject

在编写真正的内核驱动时, 我们基本上不会去直接操作 Kobject. 而是采用更上层的接口. 比如我们知道设备模型中有一个 device 的概念, 当我们需要添加一个 device 时, 调用的是系统提供的 device_add 接口, 该函数最终会去处理 Kobject 相关的事情.
我们可以理解成 device 封装了 kobject, 而这种封装动作, 在处理 kobject 的时候, 都是用的上面提到的手动方式.
所以我们在这里练习手动方式, 为的是更好的理解后面的章节(device, device_driver, bus, class, 它们才是真正的重点).

自动创建 Kobject

分析几个关键函数

kobject_create

该接口为 kobj 分配内存空间, 并以系统内置的 dynamic_kobj_ktype 为参数, 调用 kobject_init 接口，完成后续的初始化操作

kobject_create_and_add

kobject_create 和 kobject_add 的组合

DEMO

https://gitlab.com/study-linux/linux_driver_model/blob/master/auto_kobject_attribute.c

4.5 sysfs 读写属性文件的细节

前文我们介绍了 sysfs, 知道它是一种虚拟文件系统.
在介绍 Kobject 时, 我们也提及了 Attribute, 知道 Kobject 代表一个目录, 而 Attribute 代表该目录下的文件, 我们可以通过 sysfs 来读/写该属性文件.

sysfs 是一种虚拟文件系统, 虚拟文件系统也是文件系统, Linux 的文件系统都会提供 open, read, write 等接口给用户空间, 这也是 Linux 中一切皆文件的概念.

当我们想通过 sysfs 读写属性文件的时候, 首先调用的肯定是 sysfs 的 open 函数, 然后是 sysfs 的 read/write 函数. 然后呢? read/write 是如何操作到具体的属性文件的呢? struct attribute 数据结构很简单, 没有看见什么接口函数去响应 sysfs 的 read/write 接口啊!

接下来, 我们简单梳理一下这个流程:
所有的文件系统, 都会定义一个 struct file_operations 变量, 用于描述本文件系统的操作接口, sysfs 也不例外. 定义在 fs/kernfs/file.c .
下一步, 代码会进入到 fs/sysfs/file.c 中的 kernfs_ops, 有很多个 kernfs_ops, 一般针对属性文件的是 sysfs_file_kfops_rw(针对二进制属性的是 sysfs_bin_kfops_rw, 这里不细说).
下一步 sysfs_file_kfops_rw 会找到 kobject(每个属性文件都会对应某个 kobject, 当我们尝试去 open 某个属性文件时, 我们就已经知道它的 kobject 了), 获取 kobject->ktype.
下一步, ktype->sysfs_ops.

稍微总结一下, 上述流程: VFS -> file_operations -> sysfs(kernfs_ops) -> (kobject->ktype) -> (ktype->sysfs_ops).

在接下来, 如果是我们自己定义 ktype, 上述流程最终就会直接调用 ktype->sysfs_ops, 如手动创建 Kobject 的 DEMO 演示的那样;
如果是系统定义的 ktype, 则上述流程还会增加一环, (ktype->sysfs_ops) -> (kobj_attribute->show/store) , 如自动创建 Kobject 的 DEMO 演示的那样.

还有一个问题需要思考:
通过 sysfs 读属性文件, 意味着你需要往用户空间的 buffer 写入数据,通过 sysfs 写属性文件, 意味着你需要从用户空间的 buffer 读取数据.
在内核模块一文中我们讨论过, 内核不能直接访问用户空间的 buffer, 需要用 copy_to_user/copy_from_user.
如果你仔细留意了 DEMO 中的代码, 会发现我们写的 store/show 函数里面并没有去处理这个 buffer, 这是为什么?
原因是系统内部的 file_operations 这一环节帮我们处理了这些问题, 而且它还考虑了互斥的问题, 有兴趣可以看看它的代码.

4.6 kobj_attribute, device_attribute, driver_attribute, bus_attribute, class_attribute

内核中与设备模型相关的几个主要数据结构, 都存在对应的 Attribute.

它们的表现形式与 kobj_attribute 类似, 在读写属性文件的时候, 内核系统会自动帮你处理到(xxx_attribute->show/store)这一环, 你只需要用它们提供的 API(或者宏)来定义属性文件即可.

__ATTR:

```
#define __ATTR(_name, _mode, _show, _store) {\
.attr = {.name = __stringify(_name),\
.mode = VERIFY_OCTAL_PERMISSIONS(_mode) },\
.show= _show,\
.store= _store,\
}

static struct kobj_attribute foo_attribute =
        __ATTR(foo,0666, foo_show, foo_store);
```

DEVICE_ATTR

```
#define DEVICE_ATTR(_name, _mode, _show, _store) \
struct device_attribute dev_attr_##_name = __ATTR(_name, _mode, _show, _store)
```

DRIVER_ATTR

```
#define DRIVER_ATTR(_name, _mode, _show, _store) \
struct driver_attribute driver_attr_##_name = __ATTR(_name, _mode, _show, _store)

#define BUS_ATTR(_name, _mode, _show, _store)\
struct bus_attribute bus_attr_##_name = __ATTR(_name, _mode, _show, _store)

#define CLASS_ATTR(_name, _mode, _show, _store) \
struct class_attribute class_attr_##_name = __ATTR(_name, _mode, _show, _store)
```

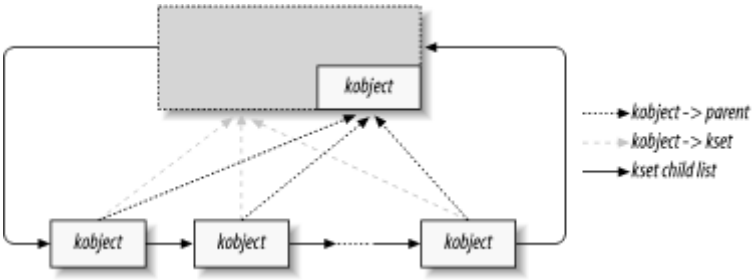
5. kset

5.1 简介

理解完 kobject, 再来看 kset 就轻松多了. 可以结合下一节的 kset 数据结构来阅读本节内容.

Kset 是一个特殊的 Kobject（因此它也会在/sys/中以目录的形式出现）, 它相当于一个容器,下面可以挂接多个 kobject. 它用来集合相似的 Kobject（这些 Kobject 可以有相同的 Ktype, 也可以有不同的 Ktype）.

前面说过, Kobject 在/sys/表现为一个目录, 这个目录的父目录是谁, 是由 kobject->parent 决定的.
当某个 kobject 属于某个 kset 时, 大多数情况下, 该 kobject->parent 会指向 kset 的 kobject. 这种情况下, 我们可以把 kset 简单看做父目录, 而属于 kset 的 kobject 都是子目录. 就像下面这张图一样:



多说一句, 注意是大多数情况下, 就算某个 kobject 属于某个 kset, 也不一定代表着该 kobject->parent 就一定指向 kset 的 kobject. 相应的, 这种情况下就不存在父/子目录的关系.

Kset 主要是跟 Uevent 事件上报相关的, Uevent 会在下一章会详解介绍.
上文提到 kset 用来集合相似的 kobject, 这个相似是指 kset 可以决定它下属的 kobject 可以上报哪些 uevent 事件; 有哪些 uevent 信息是下属所有 kobject 都要统一上报的.

5.2 主要数据结构

Kset

/* include/linux/kobject.h, line 159 */

struct kset	Comment
struct list_head list	用于链接该 kset 下所有的 kobject 的链表头

spinlock_t list_lock	锁, 用于互斥往链表中增/删 kobject
struct kobject kobj	kset 自己的 kobject（kset 是一个特殊的 kobject, 也会在 sysfs 中以目录的形式体
const struct kset_uevent_ops *uevent_ops	该 kset 的 uevent 操作函数集. 当任何 Kobject 需要上报 uevent 时, 都要调用它所 uevent_ops, 添加环境变量, 或者过滤 event（kset 可以决定哪些 event 可以上报） kobject 不属于任何 kset 时, 是不允许发送 uevent 的

kset_uevent_ops

/* include/linux/kobject.h, line 123 */

struct kset_uevent_ops	Comment
int (* const filter)(struct kset *kset, struct kobject *kobj)	filter, 当任何 Kobject 需要上报 uevent 时, 它所属的 kset 可以通过该接口过滤, event, 从而达到从整体上管理的目的
const char *(* const name)(struct kset *kset, struct kobject *kobj)	name, 该接口可以返回 kset 的名称. 如果一个 kset 没有合法的名称, 则其下的所有 上报 uevent
int (* const uevent)(struct kset *kset, struct kobject *kobj, struct kobj_uevent_env *env);	uevent, 当任何 Kobject 需要上报 uevent 时, 它所属的 kset 可以通过该接口统一为 境变量. 因为很多时候上报 uevent 时的环境变量都是相同的, 因此可以由 kset 统 每个 Kobject 独自添加了

kobj_uevent_env 就是表示具体的事件, 为在 Uevent 一章中详解解释

5.3 主要 API 说明

Kset 是一个特殊的 kobject, 因此其初始化、注册等操作也会调用 kobject 的相关接口, 除此之外, 会有它特有的部分. 另外, 和 Kobject 一样, kset 的内存分配, 可以由上层软件通过 kmalloc 自行分配, 也可以由 kset 模块负责分配, 具体如下

kset

头文件: include/linux/kobject.h, line 166

实现文件: lib/kobject.c

kset API	Comment
extern void kset_init(struct kset *kset)	该接口用于初始化已分配的 kset, 主要包括调用 kobject_init_internal 初始化 始化 kset 的链表头和锁. 需要注意的是, 如果使用此接口, 上层软件必须提供该 kset 中的 kobject 的 k
extern int __must_check kset_register(struct kset *kset)	先调用 kset_init 然后调用 kobject_add_internal 将其 kobject 添加到 kernel 然后发送一个 KOBJ_ADD uevent 事件
extern void kset_unregister(struct kset *kset)	直接调用 kobject_put 释放该 kset 的 kobject. 当其 kobject 的引用计数为 0 时 release 接口释放 kset 占用的空间
extern struct kset * __must_check kset_create_and_add(const char *name, const struct kset_uevent_ops *u, struct kobject *parent_kobj)	会调用内部接口 kset_create 动态创建一个 kset, 并调用 kset_register 将其注 kset_create 是一个内部接口(static 函数), 该接口使用 kzalloc 分配一个 kset 空 kset_ktype 类型的 ktype, 用于释放所有由它分配的 kset 空间

5.4 DEMO

想象这样一种场景: 我们想在/sys/下建 1 个父目录, 该父目录下会有 1 个属性文件, 1 个子目录, 子目录下会有 1 个属性文件. 可以读写这 2 个属性文件. 想一想你会怎么做?

首先, 我们需要创建一个 kset, 并把该 kset 添加到内核中, 这样/sys/下就会出现一个目录. 你可以用手动的方式创建 (kmalloc -> kset_register), 也可以用自动的方式创建 (kset_create_and_add).
然后, 自行定义一个属性文件, 添加到该目录下.
接下来, 创建一个 kobject, 让 kobject->parent=NULL, kobject->kset=kset. 然后把该 kobject 添加到内核, 这样该 kobject 就会自动变成 kset 的子目录.
最后, 调用 Attribute API, 在该 kobject 下创建一个属性文件.

DEMO: https://gitlab.com/study-

linux/linux_driver_model/blob/master/manual_kset_kobject_attribute.c

我们做了一个 DEMO, kset 是用手动方式创建的, kobject 也是采用手动的方式(只能用手动方式, 如果你去看 kobject 的 API, 就会发现, 只有通过手动方式, 才能指定 kobject 的 kset).

建议你阅读到这里时, **自己尝试**用自动创建 kset 的方式完成上述场景.

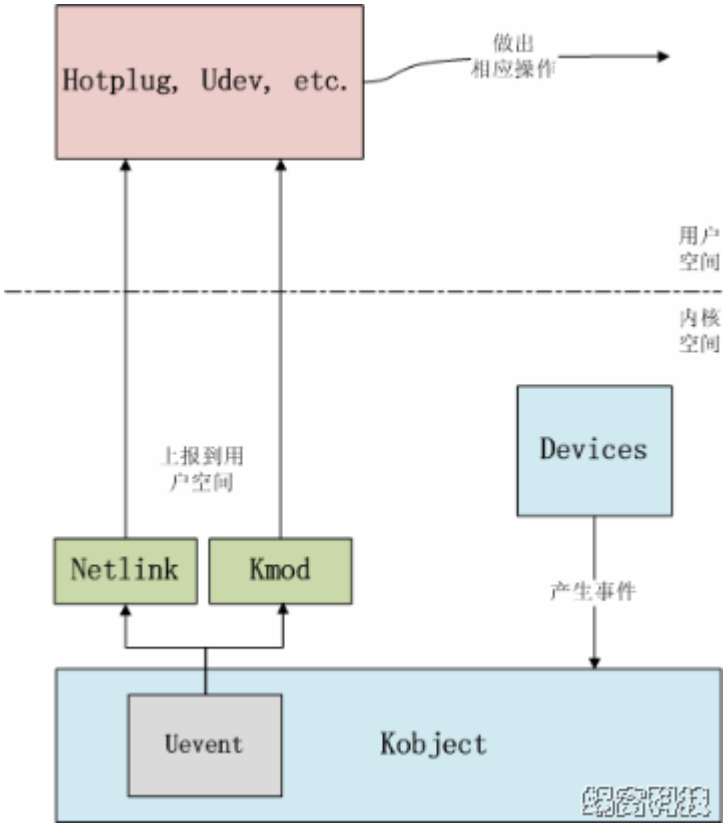
6. Uevent

6.1 简介

Uevent 是针对 Kobject 的, 用于在 Kobject 状态发生改变时, 例如增加、移除等, 通知用户空间程序. 用户空间程序收到这样的事件后, 会做相应的处理.

该机制通常是用来支持热拔插设备的, 例如 U 盘插入后, USB 相关的驱动软件会动态创建用于表示该 U 盘的 device 结构（相应的也包括其中的 kobject）, 并告知用户空间程序, 为该 U 盘动态的创建/dev/目录下的设备节点, 更进一步, 可以通知其它的应用程序, 将该 U 盘设备 mount 到系统中, 从而动态的支持该设备.

下面图片描述了 Uevent 模块在内核中的位置



由此可知, Uevent 的机制是比较简单的, 设备模型中任何设备有事件需要上报时, 会触发 Uevent 提供的接口.

Uevent 模块准备好上报事件的格式后, 可以通过两个途径把事件上报到用户空间：一种是通过 kmod 模块, 直接调用用户空间的可执行文件(简称 uevent helper); 另一种是通过 netlink 通信机制, 将事件从内核空间传递给用户空间

这两种方式可以同时存在, 也就是说只要内核同时使能了这两种方式, 一个事件会同时通过这两种方式上报.

6.2 主要数据结构

kobject_action

/* include/linux/kobject.h, line 50 */

kobject_action 定义了 event 的类型, 包括:

enum kobject_action	Comment
KOBJ_ADD	kobject 或上层数据结构(内嵌了 kobject)的添加事件

KOBJ_REMOVE	kobject 或上层数据结构(内嵌了 kobject)的移除事件
KOBJ_CHANGE	kobject 或上层数据结构(内嵌了 kobject)的状况或内容发生变化 如果设备驱动需要上报的事件不在 kobject_action 定义的事件范围内, 或者是自定义的事件, event，并携带相应的参数
KOBJ_MOVE	kobject 或上层数据结构(内嵌了 kobject)　更改名称或者更改 Parent（意味着在 sysfs 中更改
KOBJ_ONLINE	
KOBJ_OFFLINE	
KOBJ_MAX	

kobj_uevent_env

可以把 kobj_uevent_env 简单理解成一大块内存, 这块内存的 size 是 2048. 每当需要上报一个 uevent 事件时, 就会生成这样一块内存. 然后用 add_uevent_var 函数向此块内存中添加此次事件的相关信息(ACTION=%s, DEVPATH=%s, SUBSYSTEM=%s). 这些信息都是一些字符串.

添加完事件信息后, 就会把事件上报用户空间, 上一节提到过, 内核发送 uevent 事件有两种方式:

- Kmod 方式直接调用用户空间的可执行程序: 这种方式下 kobj_uevent_env 还会保存可执行程序的路径(例如/sbin/hotplug), 以及传递给可执行程序的参数. 准备好这些内容之后就会把 uevent 上报.
- netlink 方式: 这种方式下, 会把 kobj_uevent_env 中保存的事件信息 copy 到一个叫 sk_buff 的缓存中, 然后通过 socket 机制上报.

```
/* include/linux/kobject.h, line 31 */
#define UEVENT_NUM_ENVP32/* number of env pointers */
#define UEVENT_BUFFER_SIZE2048/* buffer for the variables */

/* include/linux/kobject.h, line 123 */
```

struct kobj_uevent_env	Comment
char *argv[3]	argv[0] : 保存 uevent helper 可执行的路径(例如/sbin/hotplug) argv[1] : 传递给 uevent helper 的第一个参数, 是一个字符串, 一般是 kobject 所属的 kset argv[2] : 传递给 uevent helper 的第二个参数, 一般为 NULL
char *envp[UEVENT_NUM_ENVP]	指针数组 一个事件会有多个信息, 每个信息都是一个字符串(见本节上文), 所有的信 buf[UEVENT_BUFFER_SIZE]中. envp 保存每个信息的 buf[UEVENT_BUFFER_SIZE]中的起始地址. 总共可以保存 UEVENT_NUM_ENVP 个信息的地址
int envp_idx;	每添加一个信息, envp_idx++ 记录总共添加了多少个信息
char buf[UEVENT_BUFFER_SIZE]	保存信息的 buf, 大小为 UEVENT_BUFFER_SIZE
int buflen	表示 buf 中已经用去多少空间 sizeof(buf) – buflen 就是剩余空间大小

kset_uevent_ops

kset 量身订做的一个数据结构, 方便 kset 统一管理下属 kobject 的 uevent 事件.
具体细节已经放在上文 kset 的主要数据结构中介绍了

6.3 主要 API 说明

Uevent

头文件 : include/linux/kobject.h, line 214

实现文件: lib/kobject_uevent.c

Uevent API	Comment
int kobject_uevent(struct kobject *kobj, enum kobject_action action)	针对某个 kobject 发送 kobject_action 类型的事件. 记住, 该 API 的参数是一个 kobject, 就是说只能针对某个 kobject 发送 uevent
int kobject_uevent_env(struct kobject *kobj, enum kobject_action action, char *envp[])	封装上一个 API, 不同的是, 该 API 会附加一些事件信息, 事件信息存储在 env
int add_uevent_var(struct kobj_uevent_env *env, const char *format, ...);	注意它的第一个参数, kobj_uevent_env, 就是我们在数据结构中介绍的存储事 该 API 可以理解成操作内存块的接口, 用于把事件信息(就是后面的参数: 字符 中.
int kobject_action_type(const char *buf, size_t count, enum kobject_action *type)	将 enum kobject_action 类型的 Action 转换为字符串

6.4 关键代码分析

kobject_uevent_env : 以 envp 为环境变量, 上报一个指定 action 的 uevent, 具体动作如下:

- 查找 kobj 本身或者其 parent 是否从属于某个 kset, 如果不是, 则报错返回（由此可以说明, 如果一个 kobject 没有加入 kset, 是不允许上报 uevent 的）
- 查看 kobj->uevent_suppress 是否设置, 如果设置, 则忽略所有的 uevent 上报并返回（由此可知, 可以通过 Kobject 的 uevent_suppress 标志, 管控 Kobject 的 uevent 的上报）
- 如果所属的 kset 有 uevent_ops->filter 函数, 则调用该函数, 过滤此次上报（回头查看 kset_uevent_ops 数据结构, 这佐证了有关 filter 接口的说明, kset 可以通过 filter 接口过滤不希望上报的 event, 从而达到整体的管理效果）
- 判断所属的 kset 是否有合法的名称（称作 subsystem, 和前期的内核版本有区别）, 否则不允许上报 uevent
- 分配一个用于此次上报的、存储环境变量的 buffer（也就是分配一个 kobj_uevent_env 数据结构）, 并获得该 Kobject 在 sysfs 中路径信息（用户空间程序需要依据该路径信息在 sysfs 中访问它）
- 调用 add_uevent_var 接口（下面会介绍）, 将 Action、路径信息、subsystem 等信息, 添加到 kobj_uevent_env 内存块中

- 如果传入的 envp 不空, 则解析传入的环境变量中, 同样调用 add_uevent_var 接口, 添加到 kobj_uevent_env 内存块中
- 如果所属的 kset 存在 uevent_ops->uevent 接口, 调用该接口, 添加 kset 统一的环境变量到 kobj_uevent_env 内存块中
- 根据 ACTION 的类型, 设置 kobj->state_add_uevent_sent 和 kobj->state_remove_uevent_sent 变量, 以记录正确的状态
- 调用 add_uevent_var 接口, 添加格式为“SEQNUM=%llu”的序列号
- 如果定义了“CONFIG_NET”, 则使用 netlink 发送该 uevent
- 如果定义了”CONFIG_UEVENT_HELPER”, 则调用 init_uevent_argv 初始化 kobj_uevent_env 的 argv 变量(回看 kobj_uevent_env 数据结构), 即指定 uevent helper 路径, 参数; 然后往 kobj_uevent_env 内存块中添加一些环境变量信息(HOME=/, PATH=/sbin:/bin:/usr/sbin:/usr/bin).

  最后, 用 kmod 模块提供的 call_usermodehelper 函数, 上报 uevent

  其中 uevent helper 可执行程序的路径是由内核配置项 CONFIG_UEVENT_HELPER_PATH(位于 ./drivers/base/Kconfig) 决定的 (可参考 lib/kobject_uevent.c, line 32), 该配置项指定了一个用户空间程序（或者脚本）, 用于解析上报的 uevent, 例如“/sbin/hotplug”

  call_usermodehelper 的作用, 就是 fork 一个进程, 以 uevent 为参数, 执行 uevent_helper

kobject_uevent : 和 kobject_uevent_env 功能一样, 只是没有指定任何的附加事件信息

add_uevent_var : kobj_uevent_env 内存块操作接口, 以格式化字符的形式（类似 printf、printk 等）, 将环境变量 copy 到内存块中

6.5 关于如何指定 uevent helper 可执行程序

上面介绍 kobject_uevent_env 的内部动作时, 有提到, Uevent 模块通过 Kmod 上报 Uevent 时, 会通过 call_usermodehelper 函数, 调用用户空间的可执行文件（或者脚本, 简称 uevent helper ）处理该 event. 而该 uevent helper 的路径保存在 uevent_helper 数组中.

可以在编译内核时, 通过 CONFIG_UEVENT_HELPER_PATH 配置项, 静态指定 uevent helper.但这种方式会为每个 event fork 一个进程, 随着内核支持的设备数量的增多, 这种方式在系统启

动时将会是致命的（可以导致内存溢出等）. 因此只有在早期的内核版本中会使用这种方式, 现在内核不再推荐使用该方式. **因此内核编译时, 需要把该配置项留空**

在系统启动后, 大部分的设备已经 ready, 可以根据需要, 重新指定一个 uevent helper, 以便检测系统运行过程中的热拔插事件. 这可以通过把 helper 的路径写入到“/sys/kernel/uevent_helper”文件中实现.
实际上, 内核通过 sysfs 文件系统的形式, 将 uevent_helper 数组开放到用户空间, 供用户空间程序修改访问, 具体可参考“./kernel/ksysfs.c” 中相应的代码, 这里不再详细描述

7. /sys/下的顶层目录

前面我们已经介绍了 Kobjet, Kset. 知道 Kobject 代表/sys/下的一个目录, 用于组织层次关系; 也知道 kset 是特殊的 Kobject, 因此也是/sys/下的一个目录, 用于集合有相似功能的 Kobject(主要是 Uevent 事件的处理).

下面我们要介绍一下/sys/下的一些顶层目录, 你暂时不用去关心每个目录的作用, 只需要结合前面学到的知识, 了解每个目录是如何创建的即可.

/sys/devices

/sys/dev

/sys/dev/block

/sys/dev/char

File : drivers/base/core.c

Function:

```
int __init devices_init(void)
{
devices_kset = kset_create_and_add(“devices“, &device_uevent_ops, NULL);
if (!devices_kset)
```

```
return -ENOMEM;
dev_kobj = kobject_create_and_add("dev", NULL);
if (!dev_kobj)
goto dev_kobj_err;
sysfs_dev_block_kobj = kobject_create_and_add("block", dev_kobj);
if (!sysfs_dev_block_kobj)
goto block_kobj_err;
sysfs_dev_char_kobj = kobject_create_and_add("char", dev_kobj);
……
}
```

注意该函数用__init(内核模块一文在介绍加载函数时, 介绍过该标志)标示, 代表它是在系统初始化阶段被调用的.

/sys/bus

/sys/devices/system

File : drivers/base/bus.c

Function:

```
int __init buses_init(void)
{
bus_kset = kset_create_and_add("bus", &bus_uevent_ops, NULL);
if (!bus_kset)
return -ENOMEM;

system_kset = kset_create_and_add("system", NULL, &devices_kset->kobj);
if (!system_kset)
return -ENOMEM;

return 0;
}
```

同样, __init 标示 buses_init.

system_kset 在创建时指定了 parent, 所以它位于/sys/devices 目录下.

/sys/class

File : drivers/base/class.c

Function:

```
int __init classes_init(void)
{
class_kset = kset_create_and_add("class", NULL, NULL);
if (!class_kset)
return -ENOMEM;
return 0;
}
```

/sys/kernel

File : kernel/ksysfs.c

Function:

```
static int __init ksysfs_init(void)
{
int error;

kernel_kobj = kobject_create_and_add("kernel", NULL);
if (!kernel_kobj) {
error = -ENOMEM;
goto exit;
}
……
}
```

同样, __init 标示 ksysfs_init.

/sys/module

File : kernel/params.c

Function:

```
static int __init param_sysfs_init(void)
```

```
{
module_kset = kset_create_and_add("module", &module_uevent_ops, NULL);
if (!module_kset) {
printk(KERN_WARNING "%s (%d): error creating kset\n",
__FILE__, __LINE__);
return -ENOMEM;
}
……
}
subsys_initcall(param_sysfs_init);
```

同样, __init 标示 param_sysfs_init.

/sys/block

/sys/block 其实是一个 class, 内核系统定义了一个名为"block"的 class, 当把该 class 注册进内核时, 就会产生该目录 (阅读完 Class 一章后你就会明白这个目录是如何生成的).

8. BUS

8.1 简介

总线是处理器与一个或者多个设备之间的通道, 为了方便设备模型的实现, 内核规定, 系统中的每个设备都要连接在一个 Bus 上. 这个 Bus 可以是一个内部 Bus、虚拟 Bus 或者 Platform Bus.

Bus 在内核中的作用主要体现为

- Bus 下面挂接 devices 和 drivers
- Bus 定义 device 和 driver 的匹配规则
- Bus 可以定义下属 devices 和 drivers 的默认属性

系统中有各种各样的总线, 这些总线的直观感受就是/sys/bus/下的一个目录.例如/sys/bus/spi 就代表 SPI Bus, /sys/bus/platform/代表 Platfrom Bus.

前一章我们介绍了/sys/bus 是如何生成的, 那/sys/bus 目录下的这些 Bus 是怎么生成的呢? Bus 在内核系统中到底有什么作用呢? 带着这些疑问, 开始下面的章节.

8.2 主要的数据结构

bus_type

内核通过 struct bus_type 结构抽象 Bus, 每一个 Bus 都对应一个 bus_type 结构体.

/* inlcude/linux/device.h, line 93 */

struct bus_type	Comment
const char *name	Bus 的名字 对应/sys/bus/下目录的名字.
const char *dev_name	总线下设备(struct device)的默认前缀名 当定义了一个设备, 在添加到总线下时, 没有指定设备的名称, 则总线就会帮它定 为: bus->dev_name+device ID <总线下可以挂载多个 device, 每个 device 在该总线下都有一 是总线自动分配的>
struct device *dev_root	就像每个 kobject 都可以有一个 parent, 每个 device 都可以有一个 parent device. dev_root 是该 Bus 下所有 device 的默认父设备
struct device_attribute *dev_attrs;	/* use dev_groups instead */
const struct attribute_group **bus_groups	在定义一个 bus_type 的时候, 可以指定一组"属性组", 关于属性组的概念, 可以参考 4.3 节 API: **sysfs_create_group** 当 bus 添加到内核时, 会在/sys/bus/xxx 目录下创建属性组中定义的属性文件
const struct attribute_group **dev_groups	又是一个属性组 当有 device 添加到该 Bus 下时, 会为每个 device 创建这些属性文件
const struct attribute_group **drv_groups	又是一个属性组 当有 device_driver 添加到该 Bus 下时, 会为每个 driver 创建这些属性文件

int (***match**)(struct device *dev, struct device_driver *drv)	当我们打算创建一条总线时, 我们需要定义该函数. 该函数用于定义该 Bus 下, device 和 device_driver 的匹配规则 当任何属于该 Bus 的 device 或者 device_driver 添加到该 Bus 时, 系统都会调用该函数, 该 device 和 driver 是否匹配, 如果匹配成功, 该函数要返回非零值, 此时系统就会执行后续的
int (***uevent**)(struct device *dev, struct kobj_uevent_env *env)	还记得 kset 的主要作用吗? 还记得 kset-> uevent_ops->uevent 函数的作用吗? 不记得的 当我们打算创建一条总线时, 需要定义该函数. 该函数的作用与 kset 中 uevent 函数的作用类似, 当任何属于该 Bus 的 device,发生添加、 时, Bus 模块的核心逻辑就会调用该接口, 以便添加 uevent 事件的事件信息.
int (***probe**)(struct device *dev)	当某个 device 和 driver 匹配上了之后, 系统调用 driver->probe 来初始化 driver 了. 但是, 如 实就是初始化）指定的 device 话, 需要保证该 device 所在的 bus 是被初始化过、确保能正 就在执行 device_driver 的 probe 前, 先执行它的 bus 的 probe Note: 并不是所有的 bus 都需要 probe 和 remove 接口的, 因为对有些 bus 来说（例如 plat 就是一个虚拟的总线, 无所谓初始化, 直接就能使用, 因此这个函数可以为 NULL
int (***remove**)(struct device *dev)	逻辑与上面类似.
void (***shutdown**)(struct device *dev)	与 probe 逻辑类似, 电源管理相关, 暂不说明
int (***suspend**)(struct device *dev, pm_message_t state)	与 probe 逻辑类似, 电源管理相关, 暂不说明
int (***resume**)(struct device *dev)	与 probe 逻辑类似, 电源管理相关, 暂不说明
int (*online)(struct device *dev)	热插拔相关, 如果 Bus 的 device 想支持热插拔, 则本 Bus 必须定义 online 函数
int (*offline)(struct device *dev)	热插拔相关, 如果 Bus 的 device 想支持热插拔, 则本 Bus 必须定义 offline 函数
const struct dev_pm_ops *pm	电源管理相关的逻辑, 暂不说明
struct iommu_ops *iommu_ops	暂不说明
struct subsys_private *p	很重要的一个结构, 我们专门说明

struct lock_class_key lock_key	

subsys_private

这是一个很有意思的结构体, 我们先看看它的数据结构

/* drivers/base/base.h, line 28 */

struct subsys_private	Comment
struct kset subsys	kset 在/sys/中以目录的形式存在, 代表本 Bus, 例如/sys/bus/spi, 就代表着 spi 这个目录 bus_type 中指定的
struct kset *devices_kset	kset, 代表/sys/bus/xxx/devices 这个目录 所有属于该 Bus 的 device, 都会在该目录下创建一个链接 (注意是链接哦, device 的真 的, 这里只是真实目录的一个链接, 就像我们可以在 XP 下为某个文件夹创建一个快捷方式
struct klist klist_devices	链表头, 该 Bus 下的所有 device 都会挂载到这个链表下
struct kset *drivers_kset	kset, 代表/sys/bus/xxx/drivers 这个目录 所有属于该 Bus 的 driver, 都会存放在该目录下, 注意不是链接, 是真实的目录. 意 driver->kobject 的 parent 都是该 kset->kobject
struct klist klist_drivers	链表头, 该 Bus 下的所有 drivers 都会挂载到这个链表下
struct list_head interfaces	链表头, 用于挂载该 Bus 下所有的 interfaces, interfaces 的概念暂时不详述
struct mutex mutex	与 interfaces 链表头配合, 用于互斥
struct bus_type *bus	保存上层的 Bus 指针

struct class *class	或者保存上层的 Class 指针
struct blocking_notifier_head bus_notifier	bus_notifer 通知链, 其他人需要监听该通知链时, 可以注册到该链上. 简单介绍一下内核通知链机制, 后面会有文章详述. 这种机制的做法是定义一个通知链, 然 这个通知链上是否有新动作时, 就会注册到该通知链, 当这个通知链上有动作时, 系统就会 人.
unsigned int drivers_autoprobe:1	用于控制当把 driver 添加到该 Bus 时, 是否开始匹配 driver 和 device
struct kset glue_dirs	

从数据结构中我们可以看到, 这个结构体就是集合了一些 bus 模块需要使用的私有数据, 例如 kset 啦、klist 啦等等. 那为什么这个结构体名要叫 subsys_private, 而不叫 bus_private 呢?

看看 include/linux/device.h 中的 struct class 结构（我们会在下一章中介绍 class）就知道了,因为 class 结构中也包含了一个一模一样的 struct subsys_private 指针, 看来 class 和 bus 很相似啊

想到这里, 就好理解了, 无论是 bus, 还是 class, 或者其它包含 subsys_private 的数据结构, 它们都可以理解成一个 sub-system, sub-system 就像一个独立的王国, 它下面会包含形形色色的 devices, drivers 等.
所以, 内核用 subsys_private 这样一个数据结构抽象了各种 sub-system 中的公共内容.

8.3 主要 API 说明

Bus

与 Bus 相关的 API 主要包括:

- bus 的注册和注销
- 本 bus 下有 device 或者 device_driver 注册到内核时的处理
- 本 bus 下有 device 或者 device_driver 从内核注销时的处理
- device_drivers 的 probe 处理

➢ 管理 bus 下的所有 device 和 device_driver

头文件: include/linux/device.h

实现文件: drivers/base/bus.c

bus API	Comment
extern int __must_check bus_register(struct bus_type *bus)	向系统添加一条新的 Bus
extern void bus_unregister(struct bus_type *bus)	从系统移除某一条 Bus
extern int bus_register_notifier(struct bus_type *bus, struct notifier_block *nb)	注册到 Bus 的通知链 当 Bus 上有 devices 的 addition/removal, 或者当 device 与 driver 配对/解对成 所有人都会收到通知
extern int bus_unregister_notifier(struct bus_type *bus, struct notifier_block *nb)	从 Bus 的通知链中移除 移除之后将不在收到通知

头文件: drivers/base/base.h

实现文件: drivers/base/bus.c

bus API	Comment
extern int bus_add_device(struct device *dev)	当调用 device_add 把一个 device 添加到 Bus 上时, device_add 内部会调用 在添加到 Bus 上时, Bus 这边相关的事情
extern void bus_remove_device(struct device *dev	与上相反, device 从 Bus 移除时, 调用该函数
extern void bus_probe_device(struct device *dev)	当调用 device_add 把某个 device 添加到 Bus 时, 如果 bus_set->subsys_private-> drivers_autoprobe 为 1 的话(在创建 Bus 时, 默认 Bus 的 drivers 链表下搜寻所有的 driver, 看是否有可以与 device 配对的 drive driver->probe 开始初始化

extern int bus_add_driver(struct device_driver *drv)	当调用 driver_register 把某个 driver 添加到 Bus 时, driver_register 内部会调用 driver 添加到 Bus 上时, Bus 这边相关的事情
extern void bus_remove_driver(struct device_driver *drv)	与上相反, 在 driver_unregister 时调用该函数

Driver & Device 绑定相关

头文件: include/linux/device.h

实现文件: drivers/base/dd.c

这一节的内容目前**可以跳过阅读**(现在可能还不太好理解), 在后面的章节中, 会引用到这里的相关函数, 需要了解细节的时候, 可以查阅此章节.

device_bind_driver

把某个 driver 绑定到一个 device, 在调用该函数时, device->driver 指针已经指向了某个 driver. 具体执行逻辑如下:

- **driver_sysfs_add**(struct device *dev)
  - 如果 device 属于某个 Bus, 则在 Bus 的通知链上发送消息 BUS_NOTIFY_BIND_DRIVER
  - 在 device 所指向的 driver 的目录下, 创建一个指向该 device 的链接, 链接的名称就是 device->kobj 的 name, 例如:

```
:~$ ls /sys/bus/platform/drivers/i8042/ -l
total 0
lrwxrwxrwx 1 root root    0 Aug 13 22:51 i8042 -> ../../../../devices/platform/i8042
```

  - 在该 device 的目录下, 创建一个指向该 driver 的链接, 链接的名称就是"driver"

```
$ ls /sys/devices/platform/i8042/ -l
total 0
lrwxrwxrwx 1 root root    0 Aug 13 20:13 driver -> ../../../bus/platform/drivers/i8042
```

- **driver_bound**(struct device *dev)
  - 判断该 device (dev->p->knode_driver) 有没有加入到某个 driver 的链表头
  - 如果没有, 则将它加入所指向 driver 的链表头(dev->driver->p->klist_devices), 从这里可以看到, 一个 device 只能 bound 到一个 driver, 而一个 driver 下可以挂接多个 device
  - 调用 driver_deferred_probe_trigger 处理延迟 probe 机制, 这里暂时不关注
  - 如果 device 属于某个 Bus, 则在 Bus 的通知链上发送消息 BUS_NOTIFY_BOUND_DRIVER

device_release_driver

detach device from driver.

具体执行逻辑如下:

__device_release_driver(struct device *dev)

- 获取 device->driver 所指向的 driver
- 调用 driver_sysfs_remove 删除 device_bind_driver 中创建的 2 个目录链接
- 如果 device 属于某个 Bus, 则在 Bus 的通知链上发送消息 BUS_NOTIFY_UNBIND_DRIVER
- 如果 device 属于某个 Bus 并且该 Bus 定义了 remove 函数, 则调用该 Bus 的 remove 函数; 否则如果 device->driver 所指向的 driver 定义了 remove 函数, 则调用 driver->remove
- 调用 devres_release_all 处理相关事情, 暂时不去了解细节
- 将 device->driver 和 device->driver_data 设置为 NULL
- 将 device (dev->p->knode_driver) 从 driver 的链表中删除
- 如果 device 属于某个 Bus, 则在 Bus 的通知链上发送消息 BUS_NOTIFY_UNBOUND_DRIVER

device_attach

代表有某个 device 被添加到内核, 尝试把该 device 绑定到某个 driver, 具体的执行逻辑如下:

- 如果 device->driver 已经指向了某个 driver, 并且该 device 没有添加到任何 driver 的链表头, 则调用 device_bind_driver
- 否则, 如果 device->driver 为 NULL, 则针对该 Bus 下的每一个 driver, 执行如下逻辑:
  - 首先调用 Bus 中定义的 match 函数, 判断 driver 与该 device 是否匹配. 如果不匹配则直接返回
  - 如果匹配, 则调用 driver_probe_device (在本节下面分析)

driver_attach

代表有某个 driver 被添加到内核, 尝试把该 driver 绑定到 Bus 下的一个或多个 device, 具体执行逻辑如下:

driver_attach(struct device_driver *drv)

- 针对 Bus 下的每个 device, 执行如下逻辑:

- 首先调用 Bus 中定义的 match 函数, 判断 driver 与该 device 是否匹配. 如果不匹配则直接返回
- 如果匹配, 而且该 device->driver 为 NULL, 则调用 driver_probe_device. (在本节下面分析)

driver_probe_device

driver_probe_device(struct device_driver *drv, struct device *dev)

尝试绑定 drv 和 dev, 具体指向逻辑如下:

- 首先, 判断 device 是否已经注册, 即 dev->kobj.state_in_sysfs 是否为 1, 如果不为 1, 则返回错误
- 如果为 1, 则调用 really_probe(dev, drv)处理后续事情 (在本节下面分析)

really_probe

really_probe(struct device *dev, struct device_driver *drv)

具体执行逻辑如下:

- 将 dev->driver 赋值为 drv
- 调用 pinctrl_bind_pins(dev), 该函数主要做的事情是初始化 device 相关的 GPIO, 比如我们在定义了 LCD device 的时候, 会指定该 device 用到了哪些 GPIO, 是什么复用模式, 这句代码就会自动配置这些 GPIO 为正确的状态. 这个函数依赖 GPIO 子系统, 我们会有一篇专门的文章来详述 GPIO 子系统.
- 调用 driver_sysfs_add(dev)创建一些/sys/下的目录链接, 前面有介绍这个函数
- 如果该 **Bus 定义了 probe 函数, 则调用 Bus 的 probe 函数**, 否则如果 driver 定义了 probe 函数, **则调用 driver 的 probe 函数**
- 调用 driver_bound(dev)进行处理, 前面有介绍这个函数

8.4 关键代码分析

bus_register

bus 添加到内核是由 bus_register 接口实现的, 该接口主要的功能就是**创建了一些文件夹**和一**些属性文件**, 比较简单, 执行逻辑如下:

- 为 bus_type 中 struct subsys_private 类型的指针分配空间, 并更新 priv->bus 和 bus->p 两个指针为正确的值
- 初始化通知链 priv->bus_notifier
- 接下来用手动创建 kset 的方式(可以回头看下手动创建 kset 的 DEMO), 创建 priv->subsys, 这个 kset 代表本 Bus, 即/sys/bus/xxx 中的 xxx 目录
  - 将 priv->subsys.kobj 的 name 初始化为 bus_type->name, 该 name 就是 xxx 目录的目录名
  - 将 priv->subsys.kobj 的 kset 初始化为 bus_kset, bus_kset 是系统创建的, 代表/sys/bus 这个目录(如果不记得了, 回头看下 "/sys/下的顶层目录" 一章), 由 kobject 的特性可知, 如果不指定 priv->subsys.kobj 的 parent, 则该 kset 会做为它的 parent. 实际上也是如此, 所以所有的 Bus 都会存在于/sys/bus 目录下
  - 将 priv->subsys.kobj 的 ktype 初始化为 bus_ktype, 该 ktype 也是由系统创建的. 我们自己也定义过 ktype(在手动创建 kobject 的 DEMO 中), 因此知道该 ktype 会负责释放资源和读/写 priv->subsys.kobj 下的属性文件(即/sys/bus/xxx/下的属性文件).
  - 调用 kset_register 将 priv->subsys 注册到内核中, 该调用完毕之后, 就会在/sys/bus/下生成 xxx 目录
- 调用 bus_create_file 在/sys/bus/xxx 下创建一个 uevent 属性文件.
  - 这个属性文件是系统定义的, 用 BUS_ATTR 这个宏定义, 我们在 4.6 节描述过这个宏, 回看 4.5, 4.6 两节, 你应该理解用户空间通过 sysfs 读写该属性文件的流程.
  - 不过系统创建的这个 uevent 属性文件, 只提供了写入的功能, 读函数为 NULL, 也就是说用户空间只能写入该属性文件
  - 当用户空间往该属性文件写入一个 kobject_action 类型的字符串时, 比如 KOBJ_ADD, 属性文件的写函数会调用 kobject_uevent, 针对 priv->subsys.kobj 这个 kobject, 上报一个 kobject_action 类型的 Uevent 事件, 比如上报 KOBJ_ADD 事件.
  - 至于上报这个事件有什么用, 暂时还不清楚
- 调用 kset_create_and_add 创建 priv->devices_kset 这个 kset, devices_kset 对应/sys/bus/xxx 目录下名称为"devices"的目录. Bus 下的所有 devices, 都会在该目录下创建一个链接
- 调用 kset_create_and_add 创建 priv->drivers_kset 这个 kset, drivers_kset 对应/sys/bus/xxx 目录下名称为"drivers'的目录. Bus 下的所有 drivers, 都会在该目录下创建自己的目录
- 初始化 priv 指针中的 mutex、klist_devices 和 klist_drivers 等变量
- 调用 add_probe_files 在/sys/bus/xxx 目录下创建 2 个属性文件, 这两个属性文件也是系统用 BUS_ATTR 这个宏定义的.

- 第一个属性文件是 drivers_probe, 它只有写入函数, 当用户空间往该属性文件写入一个属于该 Bus 的 device 的名称时, 会最终调用 *device_attach*, 触发系统去查找该 device 对应的 driver, 匹配上之后调用 driver->probe 初始化
- 第二个属性文件是 drivers_autoprobe, 提供了读写函数, 用于读取/写入 priv->drivers_autoprobe

➢ 调用 bus_add_groups 在/sys/bus/xxx 目录下创建 bus_type-> bus_groups 中定义的属性文件组

bus_add_device

内核提供了 device_register 接口用于把某个 device 添加到某个 Bus, device_register 会调用 device_add, device_add 会调用 bus_add_device 处理 Bus 端相关的事情

bus_add_device 的处理逻辑:

➢ 调用内部的 device_add_attrs 接口, 将由 bus->dev_attrs 指针定义的默认属性组添加到内核中, 它们会出现在 device 所在的目录下

➢ 调用内部的 device_add_groups 接口, 将由 bus-> dev_groups 指针定义的默认属性组添加到内核中, 它们会出现在 device 所在的目录下

➢ 调用 sysfs_create_link 接口, 将该 device 在 sysfs 中的目录, 链接到该 bus 的 devices 目录下, 例如

```
[redacted]:~$ ls /sys/bus/platform/devices/i8042 -l
lrwxrwxrwx 1 root root 0 Aug 13 20:13 /sys/bus/platform/devices/i8042 -> ../../.
./devices/platform/i8042
```

/sys/devices/platform/i8042 为 device 所在的真正目录, 为了方便管理, 内核在 device 所属 Bus 的 devices 目录下创建了一个符号链接, 前面在介绍 bus_type 数据结构的时候已经提到过这一点.

➢ 调用 sysfs_create_link 接口, 在该设备的 sysfs 目录中（如/sys/devices/platform/i8042/）中, 创建一个指向该设备所在 bus 目录的链接, 取名为 subsystem, 例如

```
[redacted]$ ls /sys/devices/platform/i8042/subsystem -l
lrwxrwxrwx 1 root root 0 Aug 13 20:13 /sys/devices/platform/i8042/subsystem -> .
./../../bus/platform
```

➢ 最后, 毫无疑问, 要把该 device 指针挂接到 bus->priv->klist_devices 链表头中, 这样, 就可以通过该链表头查找 Bus 下所有的 devices.

bus_add_driver

内核提供了 driver_register 接口用于把某个 driver 添加到某个 Bus, driver_register 会调用 bus_add_driver 处理 Bus 端相关的事情

bus_add_driver 的处理逻辑:

- 处理 driver-> driver_private 相关的事情, struct driver_private 会在介绍 driver 的章节详解介绍, 该结构体包含 driver 对应的 kobject, 以及一些链表头
  - 为该 driver 的 struct driver_private 指针（priv）分配空间
  - 初始化 priv->klist_devices 链表头, 该链表用于保存 driver 对应的所有 device, 也就是说一个 driver 可以与多个 device 匹配. 但是后面我们会知道, 一个 device 只能匹配一个 driver
  - 初始化 priv->driver 指向该 driver
  - 初始化 driver->p 指向该 priv
  - 将 priv->kobj.kset 赋值为 bus->p->drivers_kset, 然后调用 kobject_init_and_add 设置 kobj->ktype 为 driver_ktype, 并把该 kobj 添加到内核. 此步骤执行完毕之后, 就会在 /sys/bus/xxx/drivers 目录下生成一个 driver 对应的目录. 目录的名称是 driver->name. 例如 “/sys/bus/platform/drivers/i8042”
  - 将 priv->knode_bus 加入到 bus->p->klist_drivers 链表头, 也就是把 driver 挂接到 Bus 的 drivers 链表头下
- 如果该 driver 的 Bus 的 drivers_autoprobe 为 1, 则调用 *driver_attach*, 看看该 driver 能否和 Bus 下的 device 匹配上, 如果匹配上, 调用 driver->probe 初始化
- 调用 driver_create_file 接口, 在 sysfs 的该 driver 的目录下, 创建 uevent 属性文件, 这个属性文件是在 bus.c 中用 DRIVER_ATTR_WO(uevent)定义的. 该属性文件的作用与 /sys/bus/xxx 下的 uevent 属性文件类似, 当用户往该属性文件写入某个 kobject_action 时, 会触发系统针对该 driver 的 kobject 上报一个 kobject_action 类型的 uevent 事件.
- 调用 driver_add_attrs 接口, 在 sysfs 的该 driver 的目录, 创建由 bus->drv_attrs 指针定义的属性组
- 如果 driver->suppress_bind_attrs 为 0, 则会在 driver 的目录下创建两个属性文件
  - bind 属性文件, 在 bus.c 中用 DRIVER_ATTR_WO(bind)定义, 当用户空间写入属于该 Bus 的某个 device 的 name 时, 会调用 *driver_probe_device*, 触发系统去匹配这个 driver 和这个 device, 如果匹配成功, 会调用 driver->probe 初始化.

    这个功能与 /sys/bus/xxx 下的 drivers_probe 属性文件很像, 不同的是, 往 drivers_probe 写入 device 的 name 时, 系统会去尝试匹配该 device 与 Bus 下的所有 drivers; 而往 bind 写入 device 的 name 时, 系统只会尝试匹配 device 与该 driver

- unbind 属性文件, 在 bus.c 中用 DRIVER_ATTR_WO(unbind)定义, 与 bind 属性文件相反, 写入 device 的 name, 会调用 *device_release_driver*, 解除 device 与 driver 的绑定

bus_probe_device

内核提供了 device_register 接口用于把某个 device 添加到某个 Bus, device_register 会调用 device_add, device_add 会调用 bus_probe_device 函数.

bus_probe_device 的处理逻辑:

- 如果 Bus 的 drivers_autoprobe 为 1, 则会调用 *device_attach* 去尝试匹配该 device 和 Bus 下的某个 driver.

8.5 Demo

我们打算定义这样一个 Demo, 首先创建一条总线(Bus), 在该总线下, 创建 1 个属性文件; 然后往该 Bus 添加一个 device 和一个 driver, 观察 device 和 driver 的匹配状况.

这个 DEMO 会分几次完成, 在本章中, 我们只会创建一条 Bus, 并添加 Bus 下的属性文件, 在随后的章节中, 我们会往该 Bus 添加 device 和 driver. 你可以通过 git log 了解每一次提交的细节.
https://gitlab.com/study-linux/linux_driver_model/blob/master/bus_device_driver.c

9. Class

9.1 简介

类是一个设备的高层视图, 它抽象出了设备的共性.

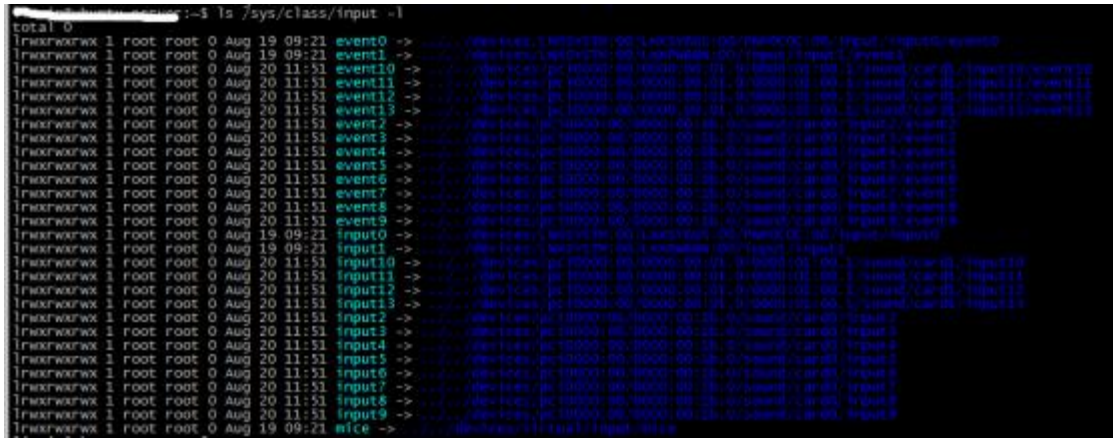
类的概念不太好理解, 举个例子: 一些年龄相仿、需要获取的知识相似的人, 聚在一起学习,就构成了一个班级（Class）. 这个班级可以有自己的名称（如 295）, 但如果离开构成它的学生（device）， 它就没有任何存在意义. 另外, 班级存在的最大意义是什么呢? 是由老师讲授的每一个课程! 因为老师只需要讲一遍, 一个班的学生都可以听到. 不然的话（例如每个学生都在家学习）, 就要为每人请一个老师, 讲授一遍. 而讲的内容, 大多是一样的, 这就是极大的浪费.

设备模型中的 Class 所提供的功能也一样了, 例如一些相似的 device（学生）, 需要向用户空间提供相似的接口（课程）, 如果每个设备的驱动都实现一遍的话, 就会导致内核有大量的冗余代码, 这就是极大的浪费. 所以, Class 说了, 我帮你们实现吧, 你们会用就行了

几乎所有的类都显示在/sys/class 目录中. 例如输入设备/sys/class/input; 串行设备/sys/class/tty. 一个例外是块设备, 出于历史原因, block class 是在/sys/block.所谓 block class 就是 name 为"block"的 class.

类成员通常被上层代码所控制, 而不需要来自驱动程序的明确支持. 极少的情况下, 驱动程序需要直接处理类

快速体验 class



- /sys/class : 除 block class, 所有的 class 都在该目录下
- /sys/class/input : input 就是一个 class
  - 在本 class 目录下, 创建每一个属于该 class 的 device 的符号链接.
    这个链接的好处是: 用户空间可以更方便的通过 sysfs 访问 device 相关的特性(也就是属性文件), 原因是路径名简短.
  - 如果 device 属于该 class, 在 device 所在的目录下, 创建一个指向本 class 的符合链接, 链接名为" subsystem", 上图没有显示, 有兴趣可以自己去看看

9.2 主要的数据结构

class

内核用 struct class 抽象一个 class, 它的定义如下

/* include/linux/device.h, line 332 */

struct class	Comment
const char *name	Class 的名字 对应/sys/class/下目录的名字.
struct module *owner	作用还不清楚
struct class_attribute *class_attrs	该 class 的默认属性, 当创建 class 时, 会在 class 的目录下(/sys/class/xxx)创建这些属
const struct attribute_group **dev_groups	属于该 class 的 device 的默认属性, 当调用 device_register 添加一个 device 时, 如果 时, 会在 device 的目录下创建这些属性文件
struct kobject *dev_kobj	表示该 class 下的设备在/sys/dev/下的目录, 现在一般有 char 和 block 两个,如果 dev 默认选择 char
int (*dev_uevent)(struct device *dev, struct kobj_uevent_env *env)	当 device 被添加到内核时, 如果 device 属于某个 class, 则会调用 class->dev_uevent 息. 我们会在讲述 device 的时候详解介绍事件上报的细节, 那里会涉及到该函数
char *(*devnode)(struct device *dev, umode_t *mode)	Callback to provide the devtmpfs 获取 path of device node file, 然后把得到的结果以 uevent 事件信息的格式上报 “
void (*class_release)(struct class *class)	用于 release 该 class 的回调函数 为什么 bus_type 里面没有一个类似的 release bus 的函数呢? 原因其实很简单, 本质上来将, 不管是 Bus 还是 Class, 都是一个 kobject, 回头看一下 说过, kobject 的释放是由它的 ktype 完成的. Bus 对应的 ktype 是 bus_ktype (bus.c); Class 对应的 ktype 是 class_ktype(class.c), bus 直接调用 kfree 释放 Bus, class_ktype->release 是调用 class->class_release 释放 Class, class_release 这个函数
void (*dev_release)(struct device *dev)	用于 release class 内设备的回调函数. 在 device_release 接口中, 会检查 Device 所在 release 接口, 如果有则调用相应的 release 接口 release 设备指针

int (*suspend)(struct device *dev, pm_message_t state)	电源管理相关
int (*resume)(struct device *dev)	电源管理相关
const struct kobj_ns_type_operations *ns_type	和 sysfs 中的名字空间有关, 这里不详述 Ktype 数据结构中也有类似的一项
const void *(*namespace)(struct device *dev)	和 sysfs 中的名字空间有关, 这里不详述 Ktype 数据结构中也有类似的一项
const struct dev_pm_ops *pm	电源管理相关
struct subsys_private *p	还记得吗, 在 Bus 一章中介绍过该数据结构, 下面我们会介绍一下 class 用到这个数 分

subsys_private

我们在 Bus 一章全面的介绍了这个数据结构, 这里只简单列出一些与 class 相关的项.

/* drivers/base/base.h, line 28 */

struct subsys_private	Comment
struct kset subsys	kset 在/sys/中以目录的形式存在, 代表本 Class, 例如/sys/class/input, 就代表着 input 这个 在 struct class 中指定的
struct kset *devices_kset	未用
struct klist klist_devices	链表头, 该 Class 下的所有 device 都会挂载到这个链表下
struct kset *drivers_kset	未用
struct klist klist_drivers	未用

struct list_head interfaces	链表头, 该 Class 下所有的 class_interface 都会挂载到这个链表下. class_interface 的作用我们在下面描述
struct mutex mutex	与 interfaces 链表头配合, 用于互斥
struct bus_type *bus	未用
struct class *class	保存上层的 Class 指针
struct blocking_notifier_head bus_notifier	未用
unsigned int drivers_autoprobe:1	未用
struct kset glue_dirs	class 有用到, 看样子应该对应一个目录, 官方的解释是为了避免 namespace conflicts, 暂时

class_interface

class_interface 的作用是: 你可以向某个 class 注册一个 class_interface (所有注册的 class_interface 都会挂载在 class->interface 链表头下), 这样当 class 下有 device 添加或移除的时候, 会调用 class_interface->add_dev 或 class_interface->remove_dev.

至于这两个函数有什么用, 就有具体的实现者决定.

/* include/linux/device.h, line 468 */

struct class_interface	Comment
struct list_head node	用于挂载到 class->interface 链表头下

struct class *class	指向所属的 class
int (*add_dev) (struct device *, struct class_interface *)	当某个 device 添加到 class 时, 会把该 device 传递给 add_dev 函数
void (*remove_dev) (struct device *, struct class_interface *)	当某个 device 从 class 移除时, 会把该 device 传递给 remove_dev 函数

9.3 主要 API 说明

Class

与 Class 相关的 API 主要包括:

- class 的注册和注销
- 向 class 注册 class_interface, 以便监听 device 的添加/移除
- 本 class 下有 device 注册到内核时的处理
- 本 class 下有 device 从内核注销时的处理

头文件: include/linux/device.h

实现文件: drivers/base/class.c

class API	**Comment**
class_register(class)	它是一个宏, 会直接调用__class_register. 用于向内核添加一个 class. 调用该 API 需要自行分配 class 内存空间
class_unregister(struct class *class)	从内核移除一个 class
class_create(owner, name)	分配 class 内存空间并把该 class 添加到内核 name 就是 class 的名称, 也就是/sys/class/xxx 的名称
class_destroy(struct class *cls)	调用 class_unregister 从内核移除一个 class. 该 API 用于销毁 class_create 创建的 class
class_interface_register(struct class_interface *)	向 class 注册一个 class_interface

class_interface_unregister(struct class_interface *)	从 class 移除一个 class_interface

9.4 关键代码分析

__class_register

class 的注册, 是由__class_register 接口实现的, 它的处理逻辑和 bus 的注册类似, 主要包括:

- 为 class 结构中的 struct subsys_private 类型的指针（cp）分配空间, 更新 cp->class 和 class->p 两个指针为正确的值
- 初始化 cp-> klist_devices 链表头
- 初始化 cp-> interfaces 链表头
- 调用 kset_init 初始化 cp->glue_dirs 这个 kset, 注意只是初始化, 但未添加到内核, 所以看不到目录
- 初始化互斥锁 cp->mutex
- 设置本 class 对应的 kobject (cp->subsys.kobj)的名字为 class->name
- 如果 class-> dev_kobj 为 NULL, 则将其设置为 sysfs_dev_char_kobj (即/sys/dev/char)
- 如果不是 block class, 则设置 cp->subsys.kobj.kset 为 class_kset (即/sys/class), 这样, 只要在把 cp->subsys.kobj 这个 kobject 添加到内核时, 设置其 parent 为 NULL, 系统就会把 kset 的 kobject 做为它的 parent.

  结果就是: 对于 block class, 目录为/sys/block, 对于其它类型的 class, 目录为/sys/class/xxx

- 设置 cp->subsys.kobj.ktype 为 class_ktype, 该 ktype 是由系统创建的 (class.c). 因此知道该 ktype 会负责释放资源和读/写 cp->subsys.kobj 下的属性文件(即/sys/class/xxx/下的属性文件)
- 调用 kset_register 初始化 cp->subsys 这个 kset 并把它添加到内核.
- 调用 add_class_attrs 把 class-> class_attrs 中定义的默认属性添加到/sys/class/xxx/目录下

__class_create

该函数的原型是

```
struct class *__class_create(struct module *owner, const char *name,
    struct lock_class_key *key)
```

从这个原型中可以看出几个重点:

- 主要的参数是 class 的 name 和 module
- 返回结果是一个 class

一句话总结: 给定一个 class 的名称和 module 参数, 它就能帮你创建一个 class.

具体的逻辑如下:

- 首先分配一个 struct class 的内存空间
- 初始化 class->name 和 class->module
- 初始化 class->class_release 为 class_create_release, class_create_release 是系统定义的 (class.c), 它其实就是用 kfree 释放第一步分配的内存空间
- 调用__class_register 注册该 class

这里想多说几句, __class_create 相当于给出一个例子, 告诉你如何手动创建一个最基础的 class. 你也可以在它的基础上, 设计复杂一点的 class, 比如设置 class 的默认属性, 设计 **dev_release** 函数, 设计 suspend/resume 等电源管理相关函数等.

device_add_class_symlinks

当调用 device_register 把某个 device 添加到内核时, 如果 device->class 不为 NULL, 则会调用该函数来处理 class 端相关的事情.

Bus 中也存在一个类似的函数 bus_add_device, 不同的是 bus_add_device 是在 bus.c 中定义的, 而 device_add_class_symlinks 是在 drivers/base/core.c 中实现的.

相信你也猜到了, 该函数就是创建一个符号链接, 我们来看看具体逻辑:

- 首先, 在 device 所在目录下, 创建一个指向该 class 的符号链接, 链接名是"subsystem"
- 如果 device->parent 不为 NULL, 并且该 device 不是一个 block device, 则在 device 所在的目录下创建一个指向 parent 的链接, 链接名是"device".

  很奇怪, 这个符号链接与 class 又没什么关系, 为什么要放在这个函数里面创建呢? 我也不清楚原因
- 如果 device 所属的 class 不是 block class, 则在 class 的目录下, 创建一个指向该 device 的符合链接, 链接名是该 device 的 name
- 如果 device 所属的 class 是 block class, 则不会在/sys/block 下创建指向该 device 的符合链接. **因为对于 block device, 它的真实目录就是在/sys/block 下**.

device_add_to_class_klist_devices

额, 怎么会有这么长名字的函数? 好吧, 我是骗你的, 其实没有这个函数.

但是这个事情还是要做的, device 属于某一个 class, 肯定要把它加入到 class->klist_devices 这个链表头.

这个事情是在 device_add 函数里面做的, 直接看一下代码好了

```
if (dev->class) {
mutex_lock(&dev->class->p->mutex);
/* tie the class to the device */
klist_add_tail(&dev->knode_class,
      &dev->class->p->klist_devices);

/* notify any interfaces that the device is here */
list_for_each_entry(class_intf,
   &dev->class->p->interfaces, node)
if (class_intf->add_dev)
class_intf->add_dev(dev, class_intf);
mutex_unlock(&dev->class->p->mutex);
}
```

很简单, 两件事情

- 把 device 加入 class->klist_devices 链表头
- 调用 class->interface 上挂载的所有 class_interface 的 add_dev 函数, 告诉它们有一个 device 添加到本 class 了.

9.5 Demo

我们打算设计这样一个 Demo: 创建一个新的基本的 class, 所谓基本的意思, 就是仿造 __class_create, 只提供一些必要的功能, 你可以在此 Demo 的基础上, 创建更复杂的 class.

然后往该 class 下添加一个 device.

https://gitlab.com/study-linux/linux_driver_model/blob/master/class_device.c

10. Device

终于.., 我们开始关注到跟我们的日常工作紧密相关的东西了.

device 和 device_driver, 在前面介绍 Bus 和 Class 的时候, 我们已经无数次提到这两个概念, 接下来我们将详细介绍这两部分

10.1 简介

device 和 device_driver 是 Linux 驱动开发的基本概念, 驱动开发是思路很简单: 开发指定的软件(device_driver)来驱动指定的设备(device).
所以 device 就是用来描述一个设备的.
device_driver 就是用来描述如何让这个设备工作的.

10.2 主要的数据结构

device

/* include/linux/device.h, line 660 */

struct device	Comment
struct device*parent	该设备的父设备. parent 的作用之一是组织 device 的层次结构, 前面我们介绍过 kobject 的主要功能 的目录层次, 这个规则不变. 实际上, 经常用 device->kobject->parent = device->pa 式, 来组织 device 的层次结构
struct device_private*p	私有数据结构指针, 从这个结构体里面我们可以看到一个 device 会被挂接到哪些锁 后面专门介绍这个结构体
struct kobject kobj	本 device 对应的 kobject, 对应 sysfs 中的一个目录
const char*init_name	就像 bus, class 一样, 每个 device 都需要有一个 name, 这个 name 体现为目录名, 也 device->kobject->name. device->kobject->name 有两种方式指定: 1. 就是这里的 init_name, 最后会赋值给 device->kobject->name 2. 如果未指定 init_name, 则会把"bus->dev_name+device->id"赋值给 device->kobjec
const struct device_type *type	与 ktype&kset 之于 kobject 有的类似

	后面专门介绍一下这个结构体
struct mutexmutex	互斥锁
struct bus_type*bus	本 device 属于哪一个 Bus
struct device_driver *driver	本 device 与哪一个 device_driver 匹配上了
void*platform_data	私有数据, device 核心代码不会触碰这个数据.
void*driver_data	某个 device_driver 的实现代码, 可能会存储一些 driver 相关数据到这个地方. device 核心代码层不会触碰这个数据. 设置的接口函数: void dev_set_drvdata(struct device *dev, void *data) 或者的接口函数: void *dev_get_drvdata(const struct device *dev)
struct dev_pm_infopower	电源管理相关
struct dev_pm_domain*pm_domain	电源管理相关
struct dev_pin_info*pins	与 GPIO(PINCTRL)子系统相关的一个数据, 定义了本 device 用到了哪些 GPIO 引脚, 怎样的. 前面我们介绍过, 当 device 和 device_driver 匹配上了之后, 在 really_probe 函数里 pinctrl_bind_pins(dev)来自动配置好这些 GPIO 引脚
intnuma_node	“NUMA”功能, 暂不描述
u64*dma_mask ~ struct dev_archdataarchdata	DMA 相关, 暂不描述
struct device_node*of_node	device tree 相关, 用于通过 device tree 传递一些 device 相关信息, 比如寄存器地址 一篇文章来描述 device tree 它也会用于 device 与 driver 的匹配

struct acpi_dev_nodeacpi_node	表示高级配置和电源管理接口, 与 ACPI device 相关 它也会用于 device 与 driver 的匹配
dev_tdevt	包含主/次设备号, 当此设备号不为 0 时, 会在/sys/dev/*/目录下创建对应的目录, 目 号:此设备号”. 同时, 还会生成对应的/dev/xxx 设备节点
u32id	该 device 的 id, 可以与 bus->dev_name 结合生成 device name 这个 id 可以手动指定, 也可以由某个 bus 自动分配
spinlock_tdevres_lock struct list_headdevres_head	与 device resource management 相关. 用于自动释放 device 所占用的资源, 我们会用专门的一章来介绍 device resource m
struct klist_node knode_class	用于把本 device 挂载到 class->subsys_private-> klist_devices 链表下
struct class *class	本 device 属于哪一个 class
const struct attribute_group **groups	本 device 的属性组
void (*release)(struct device *dev)	release 本 device
struct iommu_group *iommu_group	暂不描述
bool offline_disabled:1	设置本 device 是否支持热插拔, 为 1 表示不支持热插拔
bool offline:1	热插拔相关, 标志本 device 是处于 online 状态还是 offline 状态

device_private

关于该结构体的官方解释是: Structure to hold the private to the driver core portions of the device structure.

很明显可以看到, 该结构体是 struct device 的一部分, 但是该结构里面的内容只会被设备模型核心代码使用. 我们在编写驱动程序时不会去触屏这个数据结构.

这也许就是从 struct device 中剥离出这一部分, 单独定义成一个结构体的原因把.

/* drivers/base/base.h, line 71 */

struct device_private	Comment
struct klist klist_children	链表头, 用于挂载本 device 的 children device. 也就是 children_device->parent = 本 device
struct klist_node knode_parent	用于把本 device 挂载到 parent device 的 klist_children 链表头中
struct klist_node knode_driver	用于把本 device 挂载到与它匹配的 driver 的 klist_devices 链表头中, 所这里可以看到, 一个 载多个 device. 在 8.3 节的 *device_bind_driver* 函数中有介绍这一挂载过程.
struct klist_node knode_bus	用于把本 device 挂载到所属 Bus 的 bus_type->subsys_private-> klist_devices
struct list_head deferred_probe	用于把本 device 加入 deferred_probe_list 链表中. 官方解释很清楚, 直接贴过来: entry in deferred_probe_list which is used to retry the binding of drivers which were unable to resources needed by the device; typically because it depends on another driver getting p
struct device *device	指向本 device

总结一下一个 device 会被挂载到哪些地方

- device 的 parent device 链表中
- device 对应的 driver 的链表中
- device 所属的 Bus 的链表中

额, 好像不对啊, 不是还有一个 class 吗, 没错

- device 所属的 class 的链表中, 只不过用于挂载的 entry 没有放到 device_private 结构中, 而是在 device-> knode_class

device_type

与 ktype&kset 之于 kobject 有的类似. 回想一下 ktype 和 kset 的作用, 还记得吗?

ktype 的作用是释放 kobject 资源, 定义 kobject 默认的属性文件, 定义 kobject 下属性文件的读/写函数.

kset 的作用是聚合相似的 kobject, 控制这些 kobject 的 uevent 事件上报行为, 比如哪些事件不能上报(filter), 对于能上报的事件, 添加统一的事件信息.

device_type 也干了类似的事情, 下面我们来看看它的数据结构

/* include/linux/device.h, line 501 */

struct device_type	Comment
const char *name	本 device 属于哪一种类型的 type, 例如“partitions” and “disks” ; “mouse” and “event” 如果 name 不为 NULL, 在上报的 uevent 事件中会包含 DEVTYPE 这个事件信息
const struct attribute_group **groups	默认的属性组
int (*uevent)(struct device *dev, struct kobj_uevent_env *env)	添加统一的 uevent 事件的事件信息
char *(*devnode)(struct device *dev, umode_t *mode, kuid_t *uid, kgid_t *gid)	获取 path of device node file class 数据结构中也有一个类似的函数, 它们的功能相同, 都是获取 device node file 的路径. 系统会优先调用 device_type 中的 devnode, 如果 device_type 中没有定义, 则调用所属 clas
void (*release)(struct device *dev)	用于释放资源
const struct dev_pm_ops *pm	电源管理相关

10.3 主要 API 说明

头文件: include/linux/device.h

实现文件: drivers/base/core.c

device API	Comment
extern void device_initialize(struct device *dev)	当用 kzalloc 分配了一个 struct device 内存空间后, 调用此 API 进行初始化

extern int __must_check device_add(struct device *dev)	将初始化之后的 device 添加到内核
extern void device_del(struct device *dev)	与 device_add 的动作相反
extern int __must_check device_register(struct device *dev)	先调用 device_initialize, 然后在调用 device_add 一般我们自己定义了一个 device 之后, 都会调用该 API
extern void device_unregister(struct device *dev)	先调用 device_del 然后调用 put_device, 利用 kref 机制启动释放流程
device_create_groups_vargs	它不是一个 API, 是 core.c 定义的一个内部函数. 主要作用是: 自动分配 struct device 内存空间, 初始化此内存空间, 并将 device 添加到内核 如果执行成功, 返回一个 struct device *型数据结构
extern struct device *device_create_vargs(...);	此 API 直接调用 device_create_groups_vargs
struct device *device_create(struct class *cls, struct device *parent, dev_t devt, void *drvdata, const char *fmt, ...)	此 API 把参数(**const char *fmt, ...**)转换为 va_list, 然后调用 device_create_varg 该 API 可以指定 device 所属的 class, 一般我们想自动创建一个 device 时, 会
device_create_with_groups	此 API 与 device_create 不同之处在于, 此 API 可以指定 device 的默认属性组 device-> groups
void device_destroy(struct class *class, dev_t devt)	销毁 device_create 所创建的 device

10.4 关键代码分析

device_initialize

这个 API 很简单, 就是初始化一些链表头, 互斥锁什么的, 大致逻辑如下:

- device->kobj.kset = devices_kset.

  device_kset 是 core.c 中系统创建的一个 kset, 它对应/sys/devices 目录.
- kobject_init(&dev->kobj, &device_ktype)

  device_ktype 是 core.c 中系统定义的一个 ktype
- 别的就是一些链表头什么的, 不过说了

device_add

非常重要的 API.

下面我详细分析一下该 API, 具体逻辑如下:

- 如果 device->device_private (dev->p)为空, 则调用 kzalloc 为 dev->p 分配内存空间, 并初始化 dev->p->device, dev->p->klist_children, dev->p->deferred_probe
- 如果定义了 dev->init_name, 则把 dev->init_name 赋值给 dev->kobj->name. 然后把 dev->init_name 设为 NULL. 意味着 device_add 之后, 如果想获取 device 的 name, 只能从 dev->kobj->name 获取 (通过函数 dev_name(dev)), 而不能从 dev->init_name 获取.
- 如果 dev_name(dev)依然为 NULL, 也就是说上一步中 init_name 为 NULL, 则检查 dev->bus 是否存在, 如果存在, 则把"dev->bus->dev_name+dev->id"赋值给 dev->kobj->name
- 再次检查, 如果 dev_name(dev)依然为 NULL, 也就是 init_name 为 NULL, 而且 device 也不属于某个 Bus, 则内核不允许这种情况出现, 返回错误
- 调用 get_device_parent 获取 device 的 parent->kobject, 用于组织目录层次
  - 如果 device 属于某一个 class
    - 如果所属的 class 是 block class
      - 如果 dev->parent 不为 NULL, 并且 dev->parent->class 也是 block classs, 则返回 parent->kobject, 即/sys/block/xxx/
      - 否则, 返回 block_class.p->subsys.kobj, 即/sys/block/目录

      从这里可以看出, block class 的 device 存在于/sys/block 目录或其子目录下. 而 blcok class 这个 class 的目录就是/sys/block (见 class 章节的介绍).
    - 如果所属的 class 不是 block class
      - 如果 dev->parent 不为 NULL
        - 如果 dev->parent->class 不为 NULL, 并且 dev->class->ns_type 为 NULL, 则返回 dev->parent 所在的目录
        - 否则, 在 dev->parent 所在的目录下新建一个名为"dev->class->name"的目录, 返回此目录以做为本 device 的父目录

- 如果 dev->parent 为 NULL, 则在/sys/devices/virtual/下新建一个名为”dev->class->name”的目录, 返回此目录以做为本 device 的父目录

- 否则, device 不属于某一个 class
  - 如果 dev->parent 为 NULL, 则看看本 device 是否属于某个 Bus, 如果属于某个 Bus, 则将 dev->bus->dev_root 目录做为本 device 的父目录. 我们在 Bus 一章中介绍 bus_type 数据结构时, 有提到 dev_root 是本 Bus 下所有 device 的默认父设备
  - 如果 dev->parent 不为 NULL, 则把 parent 所在的目录做为本 device 的父目录
  - 否则返回 NULL

- 如果 get_device_parent 获取到的目录不为 NULL, 则设置 dev->kobj.parent, 否则 dev->kobj.parent 就是 NULL 了,
- 调用 kobject_add 把 dev->kobj 添加到内核, 添加完成后, 就会在 dev->kobj.parent 对应的目录下创建一个”device->name” 的目录, 如果 dev->kobj.parent 为 NULL, 则会把 dev->kobj->kset 所在的目录做为父目录, 即/sys/devices.
- 如果系统中有定义 platform_notify, 则调用此函数通知有一个 device 添加到内核了
- 调用 device_create_file 在本 device 所在的目录下, 创建一个叫 **uevent** 的属性文件, 该属性文件由 DEVICE_ATTR_RW(uevent)定义
  - 当往这个属性文件写入一个 kobject_action 类型字符串时, 会触发内核针对 dev->kobj 发送一个此类型的 uevent 事件
  - 当读取该属性文件时, 会得到 dev->kobj 所属 kset->uevent_ops->uevent 里面定义的默认事件信息
- 调用 device_add_class_symlinks 处理 class 相关事情, 在 class 一章中已经介绍过该函数
- 调用 device_add_attrs 在本 device 所在的目录下创建一些属性文件
  - 如果 dev->class 不为 NULL, 则创建 dev->class-> dev_groups 定义的属性组
  - 如果 dev->device_type 不为 NULL, 则创建 device_type-> groups 定义的属性组
  - 创建 dev->groups 中定义的属性组
  - 如果 dev->bus 不为 NULL, 且 bus->offline 和 bus->online 不为 NULL, 并且 dev->offline_disabled 为 0, 则创建一个名为 **online** 的属性文件, 该属性文件由 DEVICE_ATTR_RW(online)定义.
    - 往 online 属性文件写入 true, 会导致内核执行 device_online 函数, 此函数会调用 dev->bus->online, 然后针对 dev->kobj 发送 KOBJ_ONLINE 事件, 最后设置 dev->offline = false

- 往 online 属性文件写入 false, 会导致内核执行 device_offline 函数, 此函数会调用 dev->bus->offline, 然后针对 dev->kobj 发送 KOBJ_OFFLINE 事件, 最后设置 dev->offline = true
- 读取该属性文件则会返回 dev->offline 的值.

- 调用 bus_add_device 函数处理 bus 端相关事情, 该函数在 Bus 一章中介绍过
- 调用 dpm_sysfs_add 和 device_pm_add 处理 PM 相关事情
- 如果 dev->devt 不会 0, 说明本 device 存在主/次设备号, 则
  - 在本 device 所在的目录下创建一个名为 dev 的属性文件, 该属性文件由 DEVICE_ATTR_RO(dev)定义, 读取该属性文件, 会获取到 dev->devt 的值
  - 获取本 device 在 /sys/dev 下的目录: 如果 dev->class 不为 NULL, 则从 dev->class->dev_kobj 获取 (/sys/dev/char 或 /sys/dev/block); 否则默认就是 /sys/dev/char. 然后在/sys/dev/xxx 下创建一个指向本 device 所在目录的链接, 链接名由"dev->devt"决定
  - 调用 devtmpfs_create_node 创建设备节点, 系统启动后, 会把 devtmpfs 挂载到/dev 目录下, 这样我们就能访问到/dev/xxx 了. 至于为什么要用到 devtmpfs, 请参考第 3 章中的介绍.
- 如果 device 属于某一个 Bus, 则往 Bus 通知链接上发送一个 BUS_NOTIFY_ADD_DEVICE 消息
- 调用 kobject_uevent(&dev->kobj, KOBJ_ADD)发送 uevent 事件, 后面会分析 uevent 事件发送的详解流程
- 调用 bus_probe_device(dev)启动 device 与 driver 的匹配, 该函数在 Bus 一章分析过
- 如果 dev->parent 存在, 将本 device 加入 parent 的 klist_children 链表
- 调用假想函数 device_add_to_class_klist_devices, 将本 device 加入 class 相关链表, 该假想函数在 class 一章分析过

device_create_groups_vargs

函数原型如下

```
static struct device *
device_create_groups_vargs(struct class *class, struct device *parent,
                    dev_t devt, void *drvdata,
                     const struct attribute_group **groups,
                    const char *fmt, va_list args)
```

具体代码逻辑也很简单:

- class 参数不能为 NULL, 否则返回错误. 证明该 API 必须要求指定 device 所属的 class, 否则用手动定义 struct device 的方式来添加一个 device
- 分配一个 struct device 的内存空间
- 调用 device_initialize 初始化该内存空间
- dev->devt = devt
- dev->class = class
- dev->parent = parent
- dev->groups = groups
- dev->release = device_create_release, device_create_release 是 core.c 中定义的一个函数, 直接调用 kfree(dev)
- 将 drvdata 赋值给 dev->driver_data
- 调用 kobject_set_name_vargs(&dev->kobj, fmt, args)设置 dev->kobj 的 name
- 调用 device_add(dev)

device_create

函数原型如下:

```
struct device *device_create(struct class *class, struct device *parent,
                             dev_t devt, void *drvdata, const char *fmt, …)
```

逻辑很简单:

把参数…转换为 va_list, 然后调用 device_create_groups_vargs

device_create_with_groups

函数原型如下:

```
struct device *device_create_with_groups(struct class *class,
                                         struct device *parent, dev_t devt,
                                        void *drvdata,
                                         const struct attribute_group **groups,
                                         const char *fmt, …)
```

逻辑与 device_create 一样, 唯一的不同之处在于此 API 指定了 groups.

kobject_uevent: device uevent 事件上报流程

还记得 uevent 的事件上报流程吗, 简单点来说, 就是调用 kobject_uevent 针对某一个 kobject 上报一个 uevent 事件, 该 kobject 必须属于某个 kset 才允许上报 uevent, kset 的主要作用是过滤掉一些 uevent; 针对可以上报的 uevent, 添加统一的 uevent 事件信息. 如果不记得了, 回头看看 Uevent 一章.

回到 device 上, device 是封装的 kobject, 它的上报流程遵循上述规则. 所以需要先找到 dev->kobj 所属的 kset. 在 device_initialize 代码分析中知道这个 kset 是 devices_kset.
OK, 找到 kset, 我们着重分析这个 kset 添加了哪些 uevent 事件信息, uevent 过滤功能这里就不说了.
devices_kset 是调用 dev_uevent 来添加事件信息的, 该函数的逻辑如下:

- 如果 dev->devt 不为 0, 则添加 “MAJOR=%u” , “MINOR=%u” , “DEVNAME=%s” , “DEVMODE=%#o”, “DEVUID=%u”, “DEVGID =%u” 这些事件信息
- if (dev->type && dev->type->name), 添加(“DEVTYPE=%s”, dev->type->name)事件信息
- if (dev->driver), 添加(“DRIVER=%s”, dev->driver->name)
- 调用 of_device_uevent 添加 Device Tree 相关事件信息
- if (dev->bus && dev->bus->uevent), 调用 dev->bus->uevent 添加相关事件信息
- if (dev->class && dev->class->dev_uevent), 调用 dev->class->dev_uevent 添加相关事件信息
- if (dev->type && dev->type->uevent), 调用 dev->type->uevent 添加相关事件信息

put_device: device 资源释放流程

我们知道, ktype 是负责是否 kobject 所占用的资源的, 所以对于 device 也是一样, 当调用 put_device 是 dev->kobj 的引用计数为 0 时, dev->kob 所属的 ktype 就会开始释放资源了.

所以要先找到 dev->kobj 对应的 ktype, 在 device_initialize 代码分析中知道这个 ktype 是 device_ktype.

device_ktype 的 release 函数 device_release 的逻辑如下:

- devres_release_all(dev) : 释放 device 所占用的所有 device resource, 例如 clock, 中断号等等. 还记得 device 数据结构中的 devres_head 链表头吗? 它下面挂载了本 device 所有的 device resource. 我们会有专门的一章来分析 device resource management 机制

- if (dev->release) dev->release(dev);

  else if (dev->type && dev->type->release) dev->type->release(dev);

  else if (dev->class && dev->class->dev_release) dev->class->dev_release(dev);
- kfree(device-> device_private)

10.5 Demo

针对 device 做俩个 demo:

第一个是随 Bus Demo 一起的, 把某个 device 挂载到这个 Bus 下, 然后观察 device 与 driver 的匹配过程. 这个 Demo 中, 我们手动定义一个 struct device, 然后调用 device_register.

https://gitlab.com/study-linux/linux_driver_model/blob/master/bus_device_driver.c

第二个是随 Class Demo 创建一个 device, 把它挂载到这个 Class 下, 这个 demo 中, 我们可以直接调用 device_create 来创建这个 device.

https://gitlab.com/study-linux/linux_driver_model/blob/master/class_device.c

11. Driver

11.1 简介

device 就是用来描述一个设备的.

device_driver 就是用来描述如何让这个设备工作的

额, 好像没别的好说的...

11.2 主要的数据结构

device_driver

/* include/linux/device.h, line 229*/

struct device_driver	Comment

const char *name	本 driver 的 name
struct bus_type *bus	本 driver 所属的 Bus
struct module *owner	class 中也有一个类似的数据结构, 暂不清楚作用
const char *mod_name	used for built-in modules
bool suppress_bind_attrs	disables bind/unbind via sysfs 当把 driver 添加到内核时, 会调用 bus_add_driver Bus 一章中分析过此函数, 它会根据 driver->suppress_bind_attrs 的值来决定是否 两个属性文件. 往这两个属性文件写入 device 的 name 时, 会启动 device 与 driv
const struct of_device_id *of_match_table	编写驱动代码时经常用到它, 用于指定本 driver 可以匹配的 device. 记住 device 与 driver 是否能匹配上, 是由 bus->match 决定的, 所以 bus->match 函 这个 table
const struct acpi_device_id *acpi_match_table	与上类似
int (*probe) (struct device *dev)	这个接口函数用于实现 driver 逻辑的开始, 就像一段代码的 main 函数. 当 deive 与 driver 匹配上之后, 第一个执行的就是 driver->probe. probe 函数用于硬件上的一些初始化, 为 device 获取一些 device resource, 如果哪 本 driver 不能驱动这个 device
int (*remove) (struct device *dev)	这个接口函数用于实现 driver 逻辑的结束, 当设备被移除时, 就会执行对应 drive
void (*shutdown) (struct device *dev)	电源管理相关
int (*suspend) (struct device *dev, pm_message_t state)	电源管理相关
int (*resume) (struct device *dev)	电源管理相关
const struct attribute_group **groups	driver 的默认属性组

const struct dev_pm_ops *pm	电源管理相关
struct driver_private *p	device_driver 应该也是一个 kobject 啊, 为什么没有看见 struct kobject 数据结构 driver_private 中定义的.

driver_private

/* drivers/base/base.h, line 46*/

struct driver_private	Comment
struct kobject kobj	本 device_driver 所对应的 kobject
struct klist klist_devices	链表头, 用于挂载本 driver 下所有的 device
struct klist_node knode_bus	用于把本 device_driver 挂载到某个 Bus 下
struct module_kobject *mkobj	暂时不清楚
struct device_driver *driver	指向上层的 device_driver

11.3 主要 API 说明

头文件: include/linux/device.h

实现文件: drivers/base/driver.c

device_driver API	Comment
extern int __must_check driver_register(struct device_driver *drv)	将本 device_driver 添加到某个 Bus. 额, 参数列表里面好像没有看到 Bus 啊, 怎么知道添加到哪个 Bus 呢? 没错, 之前初始化 driver->bus
extern void driver_unregister(struct device_driver *drv)	将本 device_driver 从某个 Bus 移除, 该 API 最终会调用 bus_remove_driver, 我 数, 很简单, 有兴趣可以自己阅读代码

11.4 关键代码分析

driver_register

该 API 的逻辑比较简单, 大多数重要的事情是放在 bus_add_driver 函数中做的, 该函数我们在 Bus 一章介绍过了.

该 API 具体逻辑如下:

- 首先检查 device_driver(drv)->bus 是否存在
- 然后检查以下函数是否有在 bus 和 driver 中重复定义, 一般来讲, Bus 中定义了, 会优先调用 bus->probe, 而不会在调用 driver->probe

  drv->bus->probe && drv->probe

  drv->bus->remove && drv->remove

  drv->bus->shutdown && drv->shutdown
- 检查一下 Bus 下是否已经注册了同名(drv->name)的 driver, 如果已有, 是不允许重复注册的
- 调用 bus_add_driver(drv), 该函数会分配 driver_private 空间, 初始化 drv->kobj 并将其添加到内核, 加入链表, 创建属性文件, 启动 driver 与 device 的匹配等等.
- 调用 driver_add_groups 创建 drv->groups 中定义的默认属性组
- 调用 kobject_uevent(&drv->p->kobj, KOBJ_ADD)发送 uevent 事件

kobject_uevent: driver uevent 事件上报流程

与 device 上报 uevent 事件的逻辑一样, 先找到 drv->p->kobj 的 kset.

bus_add_driver 中初始化了此 kset. 不过它很简单, 没有 kset_uevent_ops 函数, 所以 driver 在上报 uevent 事件的时候, 不会添加任何额外的事件信息

device 与 device_driver 的匹配时机汇总

这些匹配时机的细节我们在前面都介绍过, 这里只是汇总一下:

- 当调用 device_register 把某个 device 添加到内核时 (device_register->device_add->bus_probe_device 函数中分析过)
- 当调用 driver_register 把某个 driver 添加到内核是(driver_register->bus_add_driver 函数中分析过)

- 当往/sys/bus/xxx/drivers_probe 属性文件写入一个 device name 时. (bus_register 中分析过)
- 当往/sys/bus/xxx/drivers/xxx/bind 属性文件写入一个 device name 时. (bus_add_driver 函数中分析过)

11.5 Demo

此 Demo 是随 Bus Demo 一起的, 把 driver 挂载到一个 Bus 下, 然后观察 device 与 driver 的匹配过程. 这个 Demo 中, 我们手动定义一个 struct device_driver, 然后调用 driver_register 把该 driver 挂载到某个 Bus 下.

https://gitlab.com/study-linux/linux_driver_model/blob/master/bus_device_driver.c

12. Platform 系统

12.1 简介

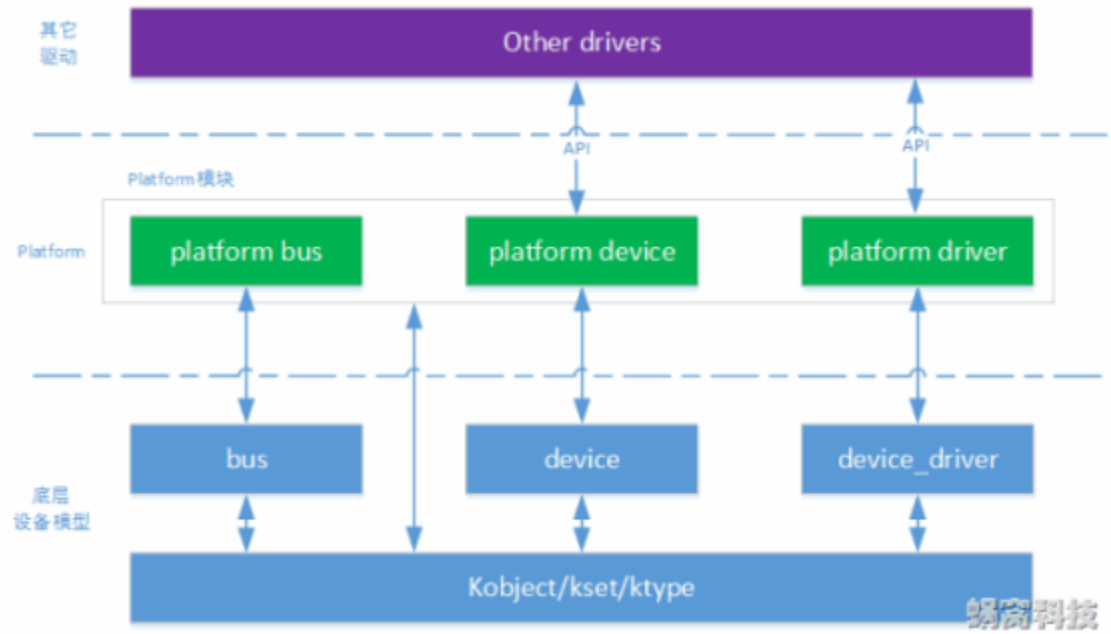
在 Linux 设备模型的抽象中, 存在着一类称作"Platform Device"的设备, 这些设备有一个基本的特征：可以通过 CPU bus 直接寻址（例如在嵌入式系统常见的"寄存器"）, 也就是我们常说的芯片内部外设, 比如 RTC, UART, LCD, SPI, I2C, 等等.

因此, 由于这个共性, 内核在设备模型的基础上（device 和 device_driver）, 对这些设备进行了更进一步的封装, 抽象出 paltform bus、platform device 和 platform driver, 以便驱动开发人员可以方便的开发这类设备的驱动.

可以说, paltform 设备对 Linux 驱动工程师是非常重要的, **因为我们编写的大多数设备驱动,都是为了驱动 plaftom 设备**. 本文我们就来看看 Platform 设备在内核中的实现.

Platform 模块的软件架构

内核中 Platform 设备有关的实现位于 include/linux/platform_device.h 和 drivers/base/platform.c 两个文件中, 它的软件架构如下



由图片可知, Platform 设备在内核中的实现主要包括三个部分:

Platform Bus, 基于底层 bus 模块, 抽象出一个虚拟的 Platform bus, 用于挂载 Platform 设备;

Platform Device, 基于底层 device 模块, 抽象出 Platform Device, 用于表示 Platform 设备;

Platform Driver, 基于底层 device_driver 模块, 抽象出 Platform Driver, 用于驱动 Platform 设备.

其中 Platform Device 和 Platform Driver 会为其它 Driver 提供封装好的 AP, 具体可参考后面的描述.

12.2 主要数据结构

platform_device

/* include/linux/platform_device.h, line 22 */

struct platform_device	Comment
struct device dev	platform_device 是对 struct device 的封装
const char *name	设备名, 用脚趾头想都知道最终会写到对应的 kobject->name
int id	对应 device->id
bool id_auto	表示 id 是不是由 platform 系统自动分配的

struct resource *resource	resource, 该设备的资源描述, 由 struct resource（include/linux/ioport.h）结构抽象 在 Linux 中, 系统资源包括 I/O、Memory、Register、IRQ、DMA、Bus 等多种类型 有独占性, 不允许多个设备同时使用, 因此 Linux 内核提供了一些 API, 用于分配、 当某个设备需要使用某些资源时, 只需利用 struct resource 组织这些资源（如名称 结束地址等）, 并保存在该设备的 resource 指针中即可.然后在设备 probe 时, 设 管理接口, 分配、使用这些资源.而内核的资源管理逻辑, 可以判断这些资源是否 被使用等等.
u32 num_resources	代表有多少个设备资源
const struct platform_device_id *id_entry	与 device 与 driver 的匹配相关. 具体可以看 platform_ bus 这个 Bus 的 match 函数
char *driver_override	Driver name to force a match
struct mfd_cell *mfd_cell	和 MFD 设备相关的内容, 暂不说明
struct pdev_archdata archdata	一个奇葩的存在！！它的目的是为了保存一些 architecture 相关的数据, 去看看 arch/arm/include/asm/device.h 中 struct pdev_archdata 结构的定义, 就知道这种东 垃圾了. 不管它了！！

platform_driver

/* include/linux/platform_device.h, line 173 */

struct platform_driver	Comment
struct device_driver driver;	platform_driver 是对 device_driver 的封装
int (*probe)(struct platform_device *)	device_driver 中定义了同名函数, 很显然, 这里的函数相当于覆盖了 device_driver 中的函数
int (*remove)(struct platform_device *)	同上
void (*shutdown)(struct platform_device *)	同上

int (*suspend)(struct platform_device *, pm_message_t state)	同上
int (*resume)(struct platform_device *)	同上
const struct platform_device_id *id_table	注意到了吗? platform_device 也有一个同类型的变量, 很明显, 它是用于 device 和 driver 的 platform_bus 的 match 函数逻辑
bool prevent_deferred_probe	延迟 probe 机制, 详解见 struct device_private 数据结构的 **deferred_probe** 这个元素的描述 从字面上的描述来看, 应该是指禁止延迟 probe 功能

platform_bus_type

platform bus 就是一个普通的 struct bus_type, 没有新增数据结构

12.3 主要 API 说明

Platform Bus

platform bus 相关的没有新增 API, 全部采用 Bus 本身定义的 API, 包括创建一个 platform bus, 销毁这个 bus 等.

Platform Device

Platform Device 主要提供设备的分配、注册等接口, 供其它 driver 使用, 具体包括:

头文件: include/linux/platform_device.h

实现文件: drivers/base/platform.c

platform device API	Comment
extern struct platform_device *platform_device_alloc(const char *name, int id);	分配一个 platform_device 结构体空间, 调用 device_init platform_device.dev , 并初始化 name, id, archdata 等元素.

extern void arch_setup_pdev_archdata(struct platform_device *);	初始化 platform_device.archdata 上一个 API 在初始化 archdata 的时候, 就是调用的此函数
extern int platform_device_add_resources(struct platform_device *pdev,const struct resource *res,unsigned int num);	往一个 platform_device 里面添加一些 resource
extern int platform_device_add_data(struct platform_device *pdev, const void *data, size_t size);	指定该 platfrom_device 的 platform data 也就是给 platfrom_device.dev.platform_data 赋值
extern int platform_device_add(struct platform_device *pdev);	将一个 paltform_device 添加到内核系统, 毫无疑问, 最终会调用 devi
extern void platform_device_del(struct platform_device *pdev);	将一个 platform_device 从内核系统移除, 主要逻辑是调用 device_del
extern void platform_device_put(struct platform_device *pdev)	调用 put_device 减少 platform_device.dev 的引用计数
extern struct platform_device *platform_device_register_full(const struct platform_device_info *pdevinfo);	总体来说, 该 API 的意思是: 给定一个 platform_device_info, 该 A platform_device 并把该 platfrom_device 添加到内核系统. 它内部会调用上面提到的 alloc, add_resources, add_data, add 等 API 注意它的第一个参数是一个结构体 platform_device_info, 这个结构主 platfrom_device 的一些信息, name, id, resources 等等
static inline struct platform_device *platform_device_register_resndata(struct device *parent, const char *name, int id, const struct resource *res, unsigned int num, const void *data, size_t size)	上一个 API 的变种, 相当于以函数参数的形式提供 name, id 等信息. 该 API 会把这些参数封装成一个 platform_device_info, 然后调用 platform_device_register_full

static inline struct platform_device *platform_device_register_simple(const char *name, int id,const struct resource *res, unsigned int num)	上一个 API 的变种, 相当于提供一些简单的参数, 然后调用 platform_device_register_resndata
static inline struct platform_device *platform_device_register_data(struct device *parent, const char *name, int id, const void *data, size_t size)	与上一个 API 的逻辑一样, 只不过参数不同而已, 最终会调用 platform_device_register_resndata
extern int **platform_device_register**(struct platform_device *)	当手动分配了一个 platform_device 结构体后, 可以调用该 API 把该 d 注意该 API 会先调用 device_initialize 初始化该 device, 所以这个 plat 用 kzlloc 分配的, 而不是用 platform_device_alloc 分配的, 因为 platfor 面也会调用 device_initialize
extern void **platform_device_unregister**(struct platform_device *)	上一个 API 的反函数
extern int **platform_add_devices**(struct platform_device **, int)	注意参数是双指针, 代表一次注册多个 platform_device 该 API 内部会调用 platform_device_register
extern struct resource *platform_get_resource(struct platform_device *, unsigned int, unsigned int)	获取 platform_device 中的 resource 信息. 第二个参数代表想要获取的 resource 的类型 (DMA/IRQ 等) 第三个参数代表想要获取的此种类型的 resource 的个数
extern int platform_get_irq(struct platform_device *, unsigned int)	与上一个 API 类似, 只不过这里类型已经确定了, 就是 IRQ.
extern struct resource *platform_get_resource_byname(struct platform_device *, unsigned int, const char *)	通过 resource 的名称获取 resource

extern int platform_get_irq_byname(struct platform_device *, const char *)	通过名称获取 IRQ

Platform Driver

Platform Driver 提供 struct platform_driver 的注册功能，没有分配函数，需要自己手动分配结构体空间，具体如下：

头文件: include/linux/platform_device.h

实现文件: drivers/base/platform.c

platform_driver API	Comment
extern int platform_driver_register(struct platform_driver *)	把一个 platform_driver 注册进内核，最终会调用 driver_register
extern void platform_driver_unregister(struct platform_driver *)	把一个 platform_driver 从内核移除，最终会调用 driver_unregister
extern int platform_driver_probe(struct platform_driver *driver,int (*probe)(struct platform_device *))	对于不可热插拔的 device，则应该用该 API 注册该 device 对应的 driver. 一般芯片内部外设都是不可热插拔的，所以芯片内部外设的驱动代码一般都 该 API 最终会调用 platform_driver_register，与 platform_driver_register 的区 热插拔设备做些处理，具体看后面的代码分析
static inline void *platform_get_drvdata(const struct platform_device *pdev)	获取 platform_device.dev.driver_data 的值 该 API 直接调用 dev_get_drvdata 可以回看 struct device 数据结构中对 driver_data 的描述
static inline void platform_set_drvdata(struct platform_device *pdev,void *data)	设置 platform_device.dev.driver_data 的值 该 API 直接调用 dev_set_drvdata 可以回看 struct device 数据结构中对 driver_data 的描述

懒人 API

又是注册 platform device, 又是注册 platform driver, 看着挺啰嗦的. 不过内核想到了这点, 所以提供一个懒人 API, 可以同时注册 platform driver, 并分配一个 platform device:

```
extern struct platform_device *platform_create_bundle(
struct platform_driver *driver, int (*probe)(struct platform_device *),
struct resource *res, unsigned int n_res,
const void *data, size_t size);
```

只要提供一个 platform_driver（要把 driver 的 probe 接口显式的传入）, 并告知该设备占用的资源信息, platform 模块就会帮忙分配资源, 并执行 probe 操作. 对于那些不需要热拔插的设备来说, 这种方式是最省事的了.

Early platform device/driver

暂时不清楚 early platform 的作用和工作机制, 后面遇到了在来补充此章节, 相关 API 包括

```
extern int early_platform_driver_register(struct early_platform_driver *epdrv,
  char *buf);
extern void early_platform_add_devices(struct platform_device **devs, int num);

static inline int is_early_platform_device(struct platform_device *pdev)
{
return !pdev->dev.driver;
}

extern void early_platform_driver_register_all(char *class_str);
extern int early_platform_driver_probe(char *class_str,
      int nr_probe, int user_only);
extern void early_platform_cleanup(void);
```

12.4 关键代码分析

platform_bus_init

很明显, platform 的第一步是要在内核系统中创建一条 platform bus, 然后才可以向该 bus 添加 platform_device 和 platform_driver

代码逻辑很简单, 如下:

- early_platform_cleanup : 为 early platform 做些准备, 这里不详述
- device_register(&platform_bus)
  - 调用 device_register 注册一个 device, 不要看参数的名字是 platform_bus, 其实它是一个 struct device 类型的.
  - 这句话的唯一作用就是在/sys/devices 下创建一个名为 platform 的目录, 所有的 platform_device 都会存放于此目录下, 所有 platform_device 的默认父设备就是 platform_bus
  - 为什么这句话能够在/sys/devices 下创建一个名为 platform 的目录呢? 如果你能很快明白, 说明前面的知识已经吸收了. platform_bus 这个 device 不属于任何 class, 也不属于任何 bus, 也没有给定 parent device, 根据前文 device_add 这个 API 的代码分析就能知道, 这个 device_register 就能在/sys/devices 下创建 platfrom 目录
- bus_register(&platform_bus_type)
  - 在内核系统中创建一条 platform Bus
  - 关于 bus_type 和 bus_register 的更多细节, 在 Bus 一章中已经介绍过了

platform_match

platform Bus 的 match 函数, 决定了该 Bus 下 platform_device (pdev)与 platform_driver(pdrv)的匹配规则, 还是比较重要的.

本节主要目的给出 pdev 与 pdrv 的匹配规则的先后顺序, 在阅读本节时可以回头多看看 pdev 与 pdrv 的数据结构, 方便理解. 具体逻辑如下:

- 如果定义了 pdev->driver_override (它是一个字符串), 则比较 pdev->driver_override 与 pdrv->driver->name, 相同就匹配, 不同就不匹配
- Attempt an OF style match
  - 判断 pdrv->driver->of_match_table 与 pdev->dev->of_node 是否能匹配上
- Then try ACPI style match
  - 判断 pdrv->driver->acpi_match_table 与 pdev->dev->acpi_node 是否能匹配上
- 如果定义了 pdrv->id_table, 则比较 pdrv->id_table.name 与 pdev->name, 相同则匹配, 不同则不匹配

- 最后一项, 回归普通的匹配规则, 判断 pdev->name 与 pdrv->driver->name 是否相同, 相同则匹配, 不同则不匹配

platform_device_register

用于把一个 platform_device（pdev）添加进内核系统.
代码逻辑很简单:

- device_initialize(&pdev->dev)
- arch_setup_pdev_archdata(pdev)
- return platform_device_add(pdev) : 这句话是关键

platform_device_add

从前文的描述中我们知道, device 属于哪个 bus 是由通过指定 device->bus 决定的. 很明显, 所有的 platform_device 都应该挂载在 platform bus 下. 照理来说, platform 系统应该自动去帮我们处理这些事情, 不需要我们手动指定 device->bus 了.

platform 系统确实是这样做的, 具体的 API 就是这个 platfrom_device_add, 逻辑如下:

- 如果 pdev->dev.parent == NULL, 则将 pdev->dev.parent 赋值为 platform_bus. 还记得 platfrom_bus 吧, 这印证了本章前文所说的, 所有 platform_device 的默认父设备都是 platform_bus
- pdev->dev.bus = &platform_bus_type : OK, 所有 platform_device 都会挂载在 platform Bus 下
- 设置 pdev->dev.kobject 的 name, 也就是 device 的 name.
  - 如果 pdev->id >= 0, 则 name 为" pdev->name.pdev->id"
  - 如果 pdev->id == PLATFORM_DEVID_NONE (-1), 则 name 为" pdev->name"
  - 如果 pdev->id == PLATFORM_DEVID_AUTO (-2), 则系统会自动获取一个 id 赋值给 pdev->id, 并把 pdev->id_auto 设为 true. name 为" pdev->name.pdev->id.auto"
- 处理 pdev 中定义的 resources, 给每个 resource 指定 name 和 parent
- device_add(&pdev->dev) : 把 device 添加到内核

platform_driver_register

把一个 platform_driver (drv) 注册进内核.

逻辑很简单, 做一些准备工作, 然后调用 driver_register. 具体逻辑如下:

- drv->driver.bus = &platform_bus_type : platform_driver 都挂载在 platform bus 下
- 如果定义了 drv->probe, 则将 platform_drv_probe 赋值给 drv->driver.probe, platform_drv_probe 是 platform.c 中定义的, 很明显, 当 device 与 driver 匹配上之后, 内核系统会调用 driver->probe, 即这里的 platform_drv_probe, 然后 platform_drv_probe 肯定会调用 drv->probe
- 如果定义了 drv->remove, 则将 platform_drv_remove 赋值给 drv->driver.remove, 逻辑与上类似, 不在赘述
- 如果定义了 drv->shutdown, 则将 platform_drv_shutdown 赋值给 drv->driver.shutdown, 逻辑与上类似
- 调用 driver_register 将 platform_driver 注册进内核

platform_driver_probe

它最终会调用 platform_driver_register, 不同之处在于该 API 是针对不可热插拔的 device, 具体逻辑如下:

- drv->prevent_deferred_probe = true : 禁止该 platform_driver 的延时 probe 机制
- drv->driver.suppress_bind_attrs = true : 不在 driver 目录下生成 bind/unbind 两个属性文件, 从而导致不可通过写入这两个属性文件来启动 device/driver 的匹配/解配, 也是为了不去处理热插拔
- drv->probe = probe (这个 probe 是本 API 的一个参数, 是个函数指针), 然后调用 platform_driver_register. 当 device 与 driver 匹配上之后, 就会调用 probe 函数
- 接下来的逻辑主要是让这个 platfrom_driver 不再尝试匹配其它 device
  - drv->probe = NULL
  - 检查 drv->driver.p->klist_devices.k_list, 如果为空, 证明这个 platfrom_driver 没有匹配到一个 device, 则返回错误
  - drv->driver.probe = platform_drv_probe_fail, 这样任何其它 device 在尝试与该 driver 匹配时, 都会返回失败

12.5 Demo

Demo 的构思也很简单, platform bus 已经存在了, 不需要重建了. 我们可以新建一个 paltform_device 和一个 platform_driver, 把它们添加到内核中, 观察两者的匹配过程.

https://gitlab.com/study-linux/linux_driver_model/blob/master/platform_device_driver.c

13. device resource management

终于, 终于, 设备模型总算是全部介绍完毕了…

本章讲一个小东西, 思路很简单, 相信很容易理解. 不过作为一个驱动工程师, 本节内容非常实用, 可以解答一些困惑, 可以使我们的代码变得简单、简洁.

13.1 前言

先看一个例子, 不用逐行看, 扫一眼即可

```
/* drivers/media/platform/soc_camera/mx1_camera.c, line 695 */

static int __init mx1_camera_probe(struct platform_device *pdev)

{

    …


    res = platform_get_resource(pdev, IORESOURCE_MEM, 0);

    irq = platform_get_irq(pdev, 0);

    if (!res || (int)irq <= 0) {

        err = -ENODEV;

        goto exit;

    }


    clk = clk_get(&pdev->dev, "csi_clk");

    if (IS_ERR(clk)) {

        err = PTR_ERR(clk);

        goto exit;

    }
```

```
pcdev = kzalloc(sizeof(*pcdev), GFP_KERNEL);

if (!pcdev) {

    dev_err(&pdev->dev, "Could not allocate pcdev\n");

    err = -ENOMEM;

    goto exit_put_clk;

}


…


/*

 * Request the regions.

 */

if (!request_mem_region(res->start, resource_size(res), DRIVER_NAME)) {

    err = -EBUSY;

    goto exit_kfree;

}


base = ioremap(res->start, resource_size(res));

if (!base) {

    err = -ENOMEM;

    goto exit_release;

}

…


/* request dma */

pcdev->dma_chan = imx_dma_request_by_prio(DRIVER_NAME, DMA_PRIO_HIGH);

if (pcdev->dma_chan < 0) {

    dev_err(&pdev->dev, "Can't request DMA for MX1 CSI\n");

    err = -EBUSY;
```

```
        goto exit_iounmap;
    }
    …

    /* request irq */
    err = claim_fiq(&fh);
    if (err) {

        dev_err(&pdev->dev, "Camera interrupt register failed\n");
        goto exit_free_dma;
    }

    …
    err = soc_camera_host_register(&pcdev->soc_host);
    if (err)
        goto exit_free_irq;

    dev_info(&pdev->dev, "MX1 Camera driver loaded\n");

    return 0;

exit_free_irq:
    disable_fiq(irq);
    mxc_set_irq_fiq(irq, 0);
    release_fiq(&fh);
exit_free_dma:
    imx_dma_free(pcdev->dma_chan);
exit_iounmap:
    iounmap(base);
exit_release:
    release_mem_region(res->start, resource_size(res));
exit_kfree:
```

```
    kfree(pcdev);
exit_put_clk:
    clk_put(clk);
exit:
    return err;
}
```

相信每一个写过 Linux driver 的工程师, 都在 probe 函数中遇到过上面的困惑: 要顺序申请多种资源（IRQ、Clock、memory、regions、ioremap、dma、等等）, 只要任意一种资源申请失败, 就要回滚释放之前申请的所有资源. 于是函数的最后, 一定会出现很多的 goto 标签（如上面的 exit_free_irq、exit_free_dma、等等）, 并在申请资源出错时, 小心翼翼的 goto 到正确的标签上, 以便释放已申请资源.

我们在内核模块一文中提到过这个现象, 那篇文章中提出了 2 种方式处理这种问题 : 使用 goto 或者 用状态标示, 然后统一在 module_exit 函数中处理.

不过, 针对 device resource, 内核 Linux 设备模型借助 device resource management（设备资源管理）, 帮我们解决了这个问题. 就是: driver 你只管申请就行了, 不用考虑释放, 我设备模型帮你释放. 最终, 我们的 driver 可以这样写:

```
static int __init mx1_camera_probe(struct platform_device *pdev)
{
    …

    res = platform_get_resource(pdev, IORESOURCE_MEM, 0);
    irq = platform_get_irq(pdev, 0);
    if (!res || (int)irq <= 0) {
        return -ENODEV;
    }

    clk = devm_clk_get(&pdev->dev, "csi_clk");
```

```
    if (IS_ERR(clk)) {
        return PTR_ERR(clk);
    }

    pcdev = devm_kzalloc(&pdev->dev, sizeof(*pcdev), GFP_KERNEL);
    if (!pcdev) {
        dev_err(&pdev->dev, "Could not allocate pcdev\n");
        return -ENOMEM;
    }

    …

    /*
     * Request the regions.
     */
    if (!devm_request_mem_region(&pdev->dev, res->start, resource_siz
e(res), DRIVER_NAME)) {
        return -EBUSY;
    }

    base = devm_ioremap(&pdev->dev, res->start, resource_size(res));
    if (!base) {
        return -ENOMEM;
    }
    …

    /* request dma */
    pcdev->dma_chan = imx_dma_request_by_prio(DRIVER_NAME, DMA_PRIO_H
IGH);
```

```
    if (pcdev->dma_chan < 0) {
        dev_err(&pdev->dev, "Can't request DMA for MX1 CSI\n");
        return -EBUSY;
    }
    …

    /* request irq */
    err = claim_fiq(&fh);
    if (err) {
        dev_err(&pdev->dev, "Camera interrupt register failed\n");
        return err;
    }

    …
    err = soc_camera_host_register(&pcdev->soc_host);
    if (err)
        return err;

    dev_info(&pdev->dev, "MX1 Camera driver loaded\n");

    return 0;
}
```

怎么做到呢?

注意上面"devm_"开头的接口, 答案就在那里. 不要再使用那些常规的资源申请接口, 用 devm_xxx 的接口代替.

为了保持兼容, 这些新接口和旧接口的参数保持一致, 只是名字前加了"devm_", 并多加一个 struct device 指针

13.2 devm_xxx

下面列举一些常用的资源申请接口, 它们由各个 framework（如 clock、regulator、gpio、等等）基于 device resource management 实现. 使用时, 直接忽略"devm_"的前缀, 后面剩下的部分, driver 工程师都很熟悉. 只需记住一点, driver 可以只申请, 不释放, 设备模型会帮忙释放. 不过如果为了严谨, 在 driver remove 时, 可以主动释放（也有相应的接口,这里没有列出）.

```
extern void *devm_kzalloc(struct device *dev, size_t size, gfp_t gfp);

void __iomem *devm_ioremap_resource(struct device *dev,
  struct resource *res);
void __iomem *devm_ioremap(struct device *dev, resource_size_t offset,
  unsigned long size);

struct clk *devm_clk_get(struct device *dev, const char *id);

int devm_gpio_request(struct device *dev, unsigned gpio,
  const char *label);

static inline struct pinctrl * devm_pinctrl_get_select(
  struct device *dev, const char *name)

static inline struct pwm_device *devm_pwm_get(struct device *dev,
  const char *consumer);

struct regulator *devm_regulator_get(struct device *dev, const char *id);

static inline int devm_request_irq(struct device *dev, unsigned int irq,
  irq_handler_t handler, unsigned long irqflags,
  const char *devname, void *dev_id);
```

```
struct reset_control *devm_reset_control_get(struct device *dev,
  const char *id);
```

13.3 什么是 device resource

一个设备能工作, 需要依赖很多的外部条件, 如供电、时钟等等, 这些外部条件称作设备资源（device resouce）. 对于现代计算机的体系结构, 可能的资源包括:

a）power : 供电

b）clock : 时钟

c）memory, 内存, 在 kernel 中一般使用 kzalloc 分配

d）GPIO, 用户和 CPU 交换简单控制、状态等信息

e）IRQ, 触发中断

f）DMA, 无 CPU 参与情况下进行数据传输

g）虚拟地址空间, 一般使用 ioremap、request_region 等分配

h）等等

而在 Linux kernel 的眼中, “资源”的定义更为广义, 比如 PWM、RTC、Reset, 都可以抽象为资源, 供 driver 使用.

在较早的 kernel 中, 系统还不是特别复杂, 且各个 framework 还没有成型, 因此大多的资源都由 driver 自行维护. 但随着系统复杂度的增加, driver 之间共用资源的情况越来越多, 同时电源管理的需求也越来越迫切. 于是 kernel 就将各个 resource 的管理权收回, 基于”device resource management”的框架, 由各个 framework 统一管理, 包括分配和回收

13.4 软件框架

说了这么久, device resource management 的软件框架到底是怎样的? 管理机制是怎么做的, 实现代码是哪个? 接下来看看这些问题.

device resource management 位于”drivers/base/devres.c”中.

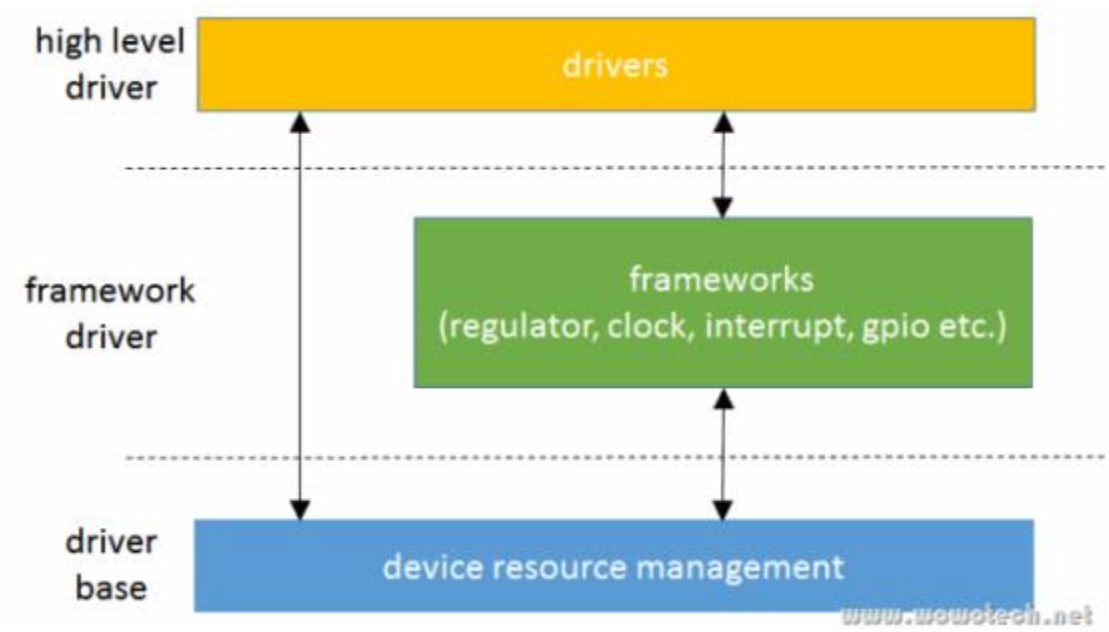
它的实现非常简单, 为什么呢? 因为资源的种类有很多, 表现形式也多种多样, 而 devres 不可能一一知情, 也就不能进行具体的分配和回收.

因此, devres 能做的（也是它的**唯一功能**）, 就是:

提供一种机制, 将系统中某个设备的所有资源, 以链表的形式, 组织起来, 以便在 driver detach（*device_release_driver*）的时候, 自动释放.

而更为具体的事情, 如怎么抽象某一种设备资源, 则由上层的 framework 负责. 这些 framework 包括: regulator framework（管理 power 资源）, clock framework（管理 clock 资源）, interrupt framework（管理中断资源）、gpio framework（管理 gpio 资源）, pwm framework（管理 PWM）, 等等.

其它的 driver, 位于这些 framework 之上, 使用它们提供的机制和接口（devm_xxx）, 开发起来就非常方便了



13.5 代码分析

相关数据结构

先从 struct device 开始吧! 还记得在介绍 struct device 数据结构时提到的 devres_head 链表头吗? 不记得了回头看看, 它就是用来挂载本 device 用到的所有 resources.

那单个 resource 用什么表示呢?

在"drivers/base/devres.c"中, 名称为 struct devres, 如下:

注意: devres 有关的数据结构, 是在 devres.c 中定义的（是 C 文件哦！）. 换句话说, 是对其它模块透明的. 这真是优雅的设计（尽量屏蔽细节）！

```
struct devres {
    struct devres_node      node;
```

```
    /* — 3 pointers */
    unsigned long long      data[]; /* guarantee ull alignment */
};
```

这个结构体很简单, 思路也很清晰: struct devres_node 描述不同 resources 相同的部分; data[] 这个零长度数组用于描述每个 resource 的不同部分 (Programming-Basics 一文中描述过零长度数组).

看一下 struct devres_node:

```
struct devres_node {
    struct list_head        entry;
    dr_release_t            release;
#ifdef CONFIG_DEBUG_DEVRES
    const char          *name;
    size_t              size;
#endif
};
```

抛开 debug 相关的不管, 也很清晰:

一个 list_head, 用于把每个 resource 挂载到 device->devres_head 链表头中.

资源的存在形式到底是什么, device resource management 并不知情, 因此需要上层模块提供一个 release 的回调函数, 用于 release 资源

API for framework driver

先看一眼软件框架里的那张图, framework driver 就是指具体某个 resource(比如 IRQ)的实现部分, 它会向某个设备驱动提供自己的 API(例如: devm_request_irq, devm_free_irq).

framework driver 是基于 device resource management 实现的, device resource management 提供给 framework driver 的 API 主要包括:

devres_alloc/devres_free、devres_add/devres_remove

结合本章前文的描述, 估计你应该能明白是怎么回事了.

先看一个使用 device resource management 的例子（IRQ 模块:devm_request_irq, devm_free_irq 是 IRQ 模块提供给设备驱动的 API, 它也用到了 resource management 提供给它的 API）：

```
/* include/linux/interrupt.h */
static inline int __must_check
devm_request_irq(struct device *dev, unsigned int irq, irq_handler_t handler,
                 unsigned long irqflags, const char *devname, void *dev_id)
{
        return devm_request_threaded_irq(dev, irq, handler, NULL, irqflags,
                                         devname, dev_id);
}


/* kernel/irq/devres.c */
int devm_request_threaded_irq(struct device *dev, unsigned int irq,
                              irq_handler_t handler, irq_handler_t thread_fn,
                              unsigned long irqflags, const char *devname,
                              void *dev_id)
{
        struct irq_devres *dr;
        int rc;

        dr = devres_alloc(devm_irq_release, sizeof(struct irq_devres),
                          GFP_KERNEL);
        if (!dr)
                return -ENOMEM;

        /*调用我们经常在驱动程序中见到的中断注册接口, 注册中断.该步骤和 device resource
management 无关.*/
```

```
        rc = request_threaded_irq(irq, handler, thread_fn, irqflags, devname,
                                  dev_id);
        if (rc) {
                devres_free(dr);
                return rc;
        }

        dr->irq = irq;
        dr->dev_id = dev_id;
        devres_add(dev, dr);

        return 0;
}
EXPORT_SYMBOL(devm_request_threaded_irq);

void devm_free_irq(struct device *dev, unsigned int irq, void *dev_id)
{
        struct irq_devres match_data = { irq, dev_id };

        WARN_ON(devres_destroy(dev, devm_irq_release, devm_irq_match,
                               &match_data));
        free_irq(irq, dev_id);
}
EXPORT_SYMBOL(devm_free_irq);
```

devres_alloc

framework driver 使用 device resource management 的第一步是:

定义一个描述自己 resource 的结构体(例如 IRQ 中的 struct irq_devres), 很显然, 那个零长度数组就会指向这个结构体的启示地址:

```
/*
```

```
* Device resource management aware IRQ request/free implementation.
*/
struct irq_devres {
        unsigned int irq;
        void *dev_id;
};
```

定义释放自己 resource 的 release 函数, 注意这个 release 不是用于释放 memory, 而是与 resource 自身相关, 例如 IRQ 资源在申请时要用 request_irq, 释放时要用 free_irq

```
static void devm_irq_release(struct device *dev, void *res)
{
        struct irq_devres *this = res;
        free_irq(this->irq, this->dev_id);
}
```

这个 release 函数会被 devres 模块调用, 注意 devres 传递过来的是 void*指针(也就是零长度数组指向的地址), 各个 framework driver 直接把 void*转换成描述自己 resource 的结构体指针即可. 对 devres 而言, 它不可能知道 resource 的具体形态, 所以只能用 void*

然后以回调函数、resource 对应的结构体的 size 为参数, 调用 devres_alloc 接口, 为 resource 分配空间. 该接口的定义如下

```
void * devres_alloc(dr_release_t release, size_t size, gfp_t gfp)
{
        struct devres *dr;

        dr = alloc_dr(release, size, gfp);
        if (unlikely(!dr))
                return NULL;
        return dr->data;
}
```

调用 alloc_dr, 分配一个 struct devres 类型的变量, 并返回其中的 data 指针（data 就是零长度数组）. alloc_dr 的定义如下:

```
static __always_inline struct devres * alloc_dr(dr_release_t release,
                                                size_t size, gfp_t gfp)
{
        size_t tot_size = sizeof(struct devres) + size;
        struct devres *dr;

        dr = kmalloc_track_caller(tot_size, gfp);
        if (unlikely(!dr))
                return NULL;

        memset(dr, 0, tot_size);
        INIT_LIST_HEAD(&dr->node.entry);
        dr->node.release = release;
        return dr;
}
```

看第一句就可以了, 在资源 size 之前, 加一个 struct devres 的 size, 就是 total 分配的空间. 除去 struct devres 的, 就是资源的（由 data 指针访问）. 之后是初始化 struct devres 变量的 node

devres_add

alloc 成功之后, 进行一下初始化, 然后以设备指针（dev）和资源指针（dr）为参数, 调用 devres_add, 将资源添加到设备的资源链表头（devres_head）中, 该接口定义如下:

```
void devres_add(struct device *dev, void *res)
{
        struct devres *dr = container_of(res, struct devres, data);
        unsigned long flags;

        spin_lock_irqsave(&dev->devres_lock, flags);
```

```
        add_dr(dev, &dr->node);
        spin_unlock_irqrestore(&dev->devres_lock, flags);
}

static void add_dr(struct device *dev, struct devres_node *node)
{
        devres_log(dev, node, "ADD");
        BUG_ON(!list_empty(&node->entry));
        list_add_tail(&node->entry, &dev->devres_head);
}
```

devres_free

可以通过 devres_free 接口释放资源占用的空间, 这里释放的是 memory:

```
void devres_free(void *res)
{
    if (res) {
        struct devres *dr = container_of(res, struct devres, data);

        BUG_ON(!list_empty(&dr->node.entry));
        kfree(dr);
    }
}
```

devres_remove

从 devres_head 中链表中移除一个资源.

```
void * devres_remove(struct device *dev, dr_release_t release,
     dr_match_t match, void *match_data)
{
```

```
struct devres *dr;
unsigned long flags;

spin_lock_irqsave(&dev->devres_lock, flags);
dr = find_dr(dev, release, match, match_data);
if (dr) {
list_del_init(&dr->node.entry);
devres_log(dev, &dr->node, "REM");
}
spin_unlock_irqrestore(&dev->devres_lock, flags);

if (dr)
return dr->data;
return NULL;
}
```

devres_destroy

devm_free_irq 接口中, 调用了 devres_destroy.

devres_destroy 主要做了 2 个动作

- 调用 devres_remove 将 devres 从 devres_head 中移除
- 调用 devres_free 释放 memory 资源

devm_free_irq 调用 devres_destroy 处理完相关事情之后, 会调用 free_irq 释放之前申请的 IRQ 资源, devm_free_irq 一般是驱动主动调用的.

API for driver model

还记得我们在前言中描述的 device resource management 的设计初衷吗? 就是会了解决在设备初始化中头疼的 goto 问题.

例如, 驱动程序的 probe 函数中可以用 request_irq 去申请一个 IRQ 资源, 当后续的初始化过程中出现错误时, 就要用 free_irq 释放这个资源.

如果在申请 IRQ 资源的时候用 devm_request_irq, 在后续初始化出错时, 可以手动用 devm_free_irq 释放资源(这种方式也要用 goto, 与上面的做法一样); 也可以什么都不管, 直接 return –XXX. driver model 的核心代码在收到这个错误之后, 会**自动释放资源**.

devres_release_all

devres_release_all 就是用来自动释放 resources 的. 实现如下:

```
int devres_release_all(struct device *dev)
{
        unsigned long flags;

        /* Looks like an uninitialized device structure */
        if (WARN_ON(dev->devres_head.next == NULL))
                return -ENODEV;
        spin_lock_irqsave(&dev->devres_lock, flags);
        return release_nodes(dev, dev->devres_head.next, &dev->devres_head,
                             flags);
}
```

以设备指针为参数, 直接调用 release_nodes:

```
static int release_nodes(struct device *dev, struct list_head *first,
                         struct list_head *end, unsigned long flags)
        __releases(&dev->devres_lock)
{
        LIST_HEAD(todo);
        int cnt;
        struct devres *dr, *tmp;

        cnt = remove_nodes(dev, first, end, &todo);

        spin_unlock_irqrestore(&dev->devres_lock, flags);
```

```
        /* Release.  Note that both devres and devres_group are
         * handled as devres in the following loop.  This is safe.
         */
        list_for_each_entry_safe_reverse(dr, tmp, &todo, node.entry) {
                devres_log(dev, &dr->node, "REL");
                dr->node.release(dev, dr->data);
                kfree(dr);
        }

        return cnt;
}
```

remove_nodes 就是从设备链表头中移除本资源.

node.release 就是调用某个资源的 release 函数, 对于 IRQ 而言, 就是调用 devm_irq_release, 这个 release 函数里面会调用 free_irq

kfree(dr)就是释放 memroy 资源.

这三步动作, 其实跟 devm_free_irq 的效果一样.

devres_release_all 的调用时机

我们知道 devres_release_all 是设备模型核心代码用来自动释放 device 所占用的 resources 的. 那核心代码在什么情况下会调用该函数呢? 有两个时机:

- probe 失败时

  调用过程为（就不详细的贴代码了）:

  __driver_attach/__device_attach–>driver_probe_device—>really_probe, really_probe 调用 driver 或者 bus 的 probe 接口, 如果失败（返回值非零, 可参考本文开头的例子）, 则会调用 devres_release_all

- driver dettach 时（就是 driver remove 时）

  driver_detach / bus_remove_device–>__device_release_driver–>devres_release_all